



How do metaheuristics exploit? A particle-level behavioral study across the CEC 2025 benchmark functions

Wulfran Fendzi Mbasso¹ · Ambe Harrison² · Zhe Liu³ · Zokir Mamadiyarov^{4,5,6} · Saad F. Al-Gahtani⁷ · Z. M. S. Elbarbary^{7,8}

Received: 23 November 2025 / Revised: 28 March 2026 / Accepted: 3 April 2026
© The Author(s), under exclusive licence to Springer-Verlag GmbH Germany, part of Springer Nature 2026

Abstract

This study presents an exploitation-aware behavioral framework for analyzing and modulating particle-level search dynamics in metaheuristic optimization. Three interpretable metrics—distance-to-reference decay, directional entropy, and stagnation indicators—are used to characterize late-stage search behavior and to trigger targeted local interventions within PSO, DE, and GWO. Experiments on the CEC 2025 benchmark suite in 10D, 30D, and 50D show that the framework provides useful behavioral diagnostics and yields function-dependent performance improvements over classical baselines. In the aggregate 30D summary, the best mean score was obtained by the EA-GWO variant, although its advantage over classical GWO was numerically small and accompanied by higher variability. Statistical analysis was revised to report significance together with effect-size interpretation, allowing statistically detectable differences to be distinguished from practically meaningful gains. The study should therefore be read primarily as a behavioral-analysis and exploitation-control contribution on representative optimizers, with broader generalization to stronger modern baselines and real-world settings left for future work.

Executive Summary: The exploitation phase is a very important part of metaheuristic optimization that people often neglect about. This is the step where algorithms improve answers that are close to the best ones they have found. A lot of research focuses on exploration and global search capacity, but particle-level exploitation dynamics remain insufficiently characterized in practice. This discrepancy has complicated the development of optimization algorithms that are more intelligent and adaptable to varying contexts. This paper presents an innovative behavioral paradigm that allows researchers to investigate exploitation dynamics in metaheuristic algorithms with remarkable precision, down to the level of individual particles. The research quantifies and clarifies the behavior of individual agents (particles, wolves, or candidates) during convergence by introducing a new set of measures, such as distance-to-best decay, local movement entropy, and standstill frequency. The method is rigorously applied to the CEC 2025 real-parameter benchmark suite, which is a

✉ Wulfran Fendzi Mbasso
fendzi.wulfran@yahoo.fr

✉ Ambe Harrison
ambe.harrison@ubuea.cm

Zhe Liu
liuzhe921@gmail.com

Zokir Mamadiyarov
zokir_mamadiyarov@tues.uz

Saad F. Al-Gahtani
saljbar@kku.edu.sa

Z. M. S. Elbarbary
albrbry@kku.edu.sa

¹ Department of Biosciences, Saveetha School of Engineering.,
Saveetha Institute of Medical and Technical Sciences,
Chennai - 602 105, Chennai, India

² Applied Science Research Center, Applied Science Private
University, Amman, Jordan

³ School of Computer Sciences, Universiti Sains Malaysia,
Penang Penang 11800, Malaysia

⁴ Department of Finance and Tourism, Termez University of
Economics and Service, Termez, Uzbekistan

⁵ Department of Economics, Mamun University, Khiva,
Uzbekistan

⁶ Department of Bank Accounting and Auditing, Tashkent
State University of Economics, Tashkent, Uzbekistan

⁷ Department of Electrical Engineering, College of
Engineering, King Khalid University, 61421 Abha, Saudi
Arabia

⁸ Center for Engineering and Technology Innovations, King
Khalid University, 61421, Abha, Saudi Arabia

challenging and current testbed for evaluating optimization algorithms. We look at a few well-known algorithms, such as GWO, DE, and PSO, as well as novel exploitation-aware versions that change how local search works based on how particles move. The results reveal that the proposed behavior-aware variants showed stronger function-dependent refinement behavior than the baseline methods. On selected difficult functions, the exploitation-aware GWO variant showed clearer late-stage refinement and function-specific improvements; however, the aggregate benchmark-level advantage over classical GWO was modest, and the revised manuscript now reports this distinction explicitly. This study's results enhance our understanding by offering: A way to judge how well an algorithm works rather than by its ultimate scores;

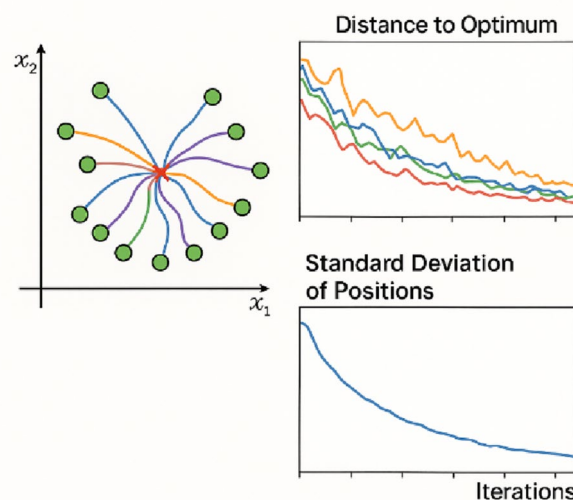
A good collection of measurements for measuring how different and similar behaviors are;

A foundation for developing next-generation, behavior-aware metaheuristics that exhibit enhanced adaptability.

This study is beneficial in both theoretical and practical contexts. It helps people learn how to use stochastic optimization and gives researchers and professionals practical tools to make algorithm creation more dependable and efficient.

Graphical abstract

A Particle-Level Behavioral Study Across the CEC 2025 Benchmark Functions



Keywords Metaheuristic optimization · Exploitation behavior · Particle dynamics · CEC 2025 benchmark · Swarm intelligence · Behavioral analysis framework

1 Introduction

1.1 Background of the study

Metaheuristic optimization techniques are now crucial for addressing numerous complex, high-dimensional, nonlinear, and multimodal challenges in science and engineering [1–3]. Some examples of strong and flexible algorithms are Particle Swarm Optimization (PSO), Differential Evolution (DE), Grey Wolf Optimizer (GWO), and Genetic Algorithms (GA). These algorithms work by imitating intelligent behaviors that happen in nature, biological evolution, or social systems. Their strength lies in their ability to balance two complimentary processes: exploration, which ensures a broad search across the issue domain, and exploitation,

which enables convergence to local or global optima through a more focused search in advantageous locations [4, 5]. There has been a lot of research on exploration and how to make it better using different diversification and adaptive control mechanisms. However, the exploitation phase, which is how an algorithm fine-tunes solutions, has not been studied as much, especially at the level of individual agents. Most studies have examined metaheuristics utilizing population-level data or aggregate fitness enhancements, mostly overlooking the behaviors and interactions of individual agents during convergence [6–8]. In many real-world circumstances, such as optimizing structures, constructing control systems, and adjusting hyperparameters for machine learning, an algorithm's failure to perform precise and consistent local search might result in suboptimal or unstable

solutions. Researchers need to understand and enhance how metaheuristics use information since optimization problems are getting harder and harder to solve.

1.2 Related works

There have been a lot of efforts in the last few years to make metaheuristics better by using adaptive mechanisms, hybridization, and ways to keep variation. For example, [9] made the Success-History Based Adaptive Differential Evolution (SHADE), which alters control parameters on the fly to help diverse CEC benchmarks come together faster. [10] also built the Grey Wolf Optimizer, which is based on how grey wolves hunt and lead. It has worked successfully on issues that require global optimization. These tactics are good for improving overall performance, but they usually focus on either making it easier for people to explore or changing parameters at the population level in a way that works best for everyone. Recent research has concentrated on regulating diversity and the behavior of swarms in proximity to optimal conditions. [11] examined the impact of dynamic parameter adjustment in differential evolution on convergence behavior. [12] proposed a diversity-preserving PSO that employs adaptive learning and mutation to prevent premature convergence. These studies predominantly utilize coarse-grained measures such as swarm variance or best-so-far trajectories, which fail to provide insights into the fine-grained dynamics of individual agents during exploitation. In addition, more and more of the benchmark suites used in earlier examinations, such as CEC 2013 and CEC 2017, are being replaced by harder and more realistic standards, such as the CEC 2025 suite [13]. Recent applications of metaheuristics in high-dimensional learning and design tasks further motivate the need for interpretable behavioral diagnostics. For example, metaheuristic-based pruning of convolutional neural networks has been systematically compared by Palakonda et al., highlighting practical exploration–exploitation trade-offs in achieving sparse yet accurate models [14]. In smart-grid settings, Palakonda et al. studied Differential Evolution with stochastic selection in uncertain environments, illustrating how DE variants must remain robust under uncertainty and shifting operating conditions [15]. Beyond energy, Differential Evolution has also been used in interactive, user-driven optimization workflows such as interior design recoloring based on YUV edge detection [16]. Many-objective real-world engineering benchmarks and state-of-the-art algorithmic comparisons have also been reported in IEEE Access, emphasizing the need for reliable exploitation under complex multi-criteria landscapes [17]. Our work is complementary: rather than proposing yet another high-level optimizer, we introduce particle-level behavioral metrics that make exploitation

dynamics measurable and actionable, which can be used to analyze and improve the exploitation behavior of such methods across domains.

1.3 Research gaps and aims of the work

There has been a lot of work done on metaheuristic algorithms, but there is still a large gap in the systematic study of how particles or agents behave when they exploit. There is limited study examining the movement patterns of individual agents within the search space, their adaptive behaviors under convergence pressure, or the variability in exploitation effectiveness based on specific issue instances and dimensions. Without this level of information, algorithm designers cannot figure out why convergence comes too fast, why stagnation zones arise, or why local instability happens during the later stages of optimization. This paper aims to address this gap by conducting a comprehensive particle-level analysis of the functioning of exploitation in metaheuristic algorithms. We use the most recent and comprehensive benchmark suite, CEC 2025, to examine the behavior of individual agents throughout the exploitation phase across a diverse array of real-parameter optimization problems. We devise and compute innovative behavioral metrics, such as distance-to-optimum decay, local movement entropy, and stagnation time, to assess agent exploitation during the optimization process. We do not just want to make algorithms easier to understand. We also want to make it easy to design popular metaheuristics that are aware of exploitation and adapt how they search based on feedback from real-time diagnostics.

1.4 Contributions and novelty of the study

This research alters the perspective on metaheuristic performance by transitioning from conventional fitness-based and population-wide evaluations to a more granular, agent-level behavioral framework emphasizing exploitation. We have made contributions to both theory and practice. First, we develop a diagnostic framework to measure how particles behave by coming up with new metrics that demonstrate how agents change, stay the same, or go their own way during the exploitation phase. Second, we apply this paradigm to established metaheuristics such as PSO, DE, and GWO, as well as to novel exploitation-aware variants that incorporate local learning, adaptive neighborhood control, or entropy-guided mutation. Third, we do a complete review of the CEC 2025 benchmark suite, which is a stringent, high-dimensional, and multimodal testbed for testing different ways to optimize real parameters. Lastly, we show that the proposed exploitation-aware algorithms showed stronger performance on selected difficult functions than their

traditional counterparts in terms of accuracy, resilience, and efficiency of computing. Our study is novel because it examines behavior, which not only enhances performance but also elucidates the reasons for these improvements based on agents' actions. This shrinks the gap between using a black-box algorithm and understanding it, which opens the door for smarter and more flexible ways to optimize.

1.5 Paper layout

The remainder of this paper is structured as follows. Section 2 introduces the theoretical foundation for exploitation behavior in metaheuristics, including formal definitions of the behavioral metrics used in this study. Section 3 describes the experimental setup, including the benchmark functions from the CEC 2025 suite, evaluation criteria, and algorithm configurations. Section 4 presents the baseline and exploitation-aware variants of the algorithms used in the analysis. Section 5 details the empirical results, offering both quantitative comparisons and qualitative insights into exploitation patterns. Section 6 discusses the broader implications of the findings, providing design guidelines and theoretical considerations for future algorithm development. Finally, Sect. 7 concludes the paper and outlines directions for future research.

2 Theoretical foundation for exploitation behavior in metaheuristics

2.1 Theoretical analysis

Exploitation behavior in metaheuristic algorithms indicates that the algorithm can quickly locate the best solutions, whether they are global or local, by improving replies in a promising area of the search space. Finding new areas and being open to new things is what exploration is all about. Exploitation, on the other hand, means looking for things just in places that are known to be high-quality. It is vital to understand this phase in order to find out how effectively convergence works, how strong it is, and how likely it is to stop operating too soon. When scholars examine how well metaheuristics work, they commonly look at summary statistics like the ultimate objective value, the speed of convergence, or measures of diversity at the population level [18]. But these kinds of measures do not show the little motions of individual actors (such particles, wolves, population, etc.), which are particularly critical for exploitation. To address this deficiency, our study introduces a particle-level analytical framework grounded in various behavioral metrics, which we elucidate and demonstrate in this section. Also, it is important to notice that we distinguish three separate

concepts. (i) Movement stagnation refers to an agent whose displacement magnitude remains below a threshold δ for consecutive iterations. (ii) Personal-best stagnation refers to an agent whose personal best has not improved for T_a iterations; this is the trigger used in EA-DE. (iii) Global stagnation refers to the absence of meaningful improvement in the population best over a rolling window, measured using tolerance η ; this is used only in the adaptive supervisory logic. These notions are related but not interchangeable, and we now state explicitly which one is used in each algorithmic component.

Let $x_i^t \in \mathbb{R}^d$ be the position of the i -th agent at iteration t , and let x^* be the position of the best known or estimated global optimum. We suggest the following metrics to measure and show how people exploit:

For benchmark functions with known shifted optimum x^* , the particle-level distance-to-optimum metric was computed as (1). The use of a known optimum location is standard in benchmark-based optimization studies and follows the benchmark-definition literature [19]:

$$D_i^t = \|x_i^t - x^*\|_2 \quad (1)$$

A rapid decay in D_i^t across iterations indicates efficient exploitation, while stagnation or oscillation suggests local trapping or ineffective refinement. On synthetic benchmarks such as the CEC 2025 suite, the true global optimum is known by construction, and x^* in (1) is set to this reference point. In real-world black-box problems, however, the optimum is unknown and may be noisy or deceptive. In these cases we replace x^* by an evolving reference $\hat{x}^*$ equal to the current best-so-far solution (or a high-quality reference solution when available). The distance-to-optimum decay metric then tracks convergence toward $\hat{x}^*$ rather than an absolute ground truth. This approximation can introduce bias when $\hat{x}^*$ is far from the true optimum or when noise occasionally degrades the best-so-far record. We therefore interpret distance decay in conjunction with directional entropy and stagnation frequency, which do not depend on the true optimum and remain well-defined in noisy or deceptive landscapes. In Sect. 6 we explicitly discuss these limitations and outline how robust reference estimates could further stabilize the metric in real-world deployments. Figure 1 shows the average distance decay for 10 agents moving toward the origin (the global optimum) in a simulated exploitation scenario.

The second metric is trajectory entropy, which shows how consistently agents travel in a certain direction during exploitation. Let the movement vector change between iterations be:

$$Dx_i^t = x_i^t - x_i^{t-1} \quad (2)$$

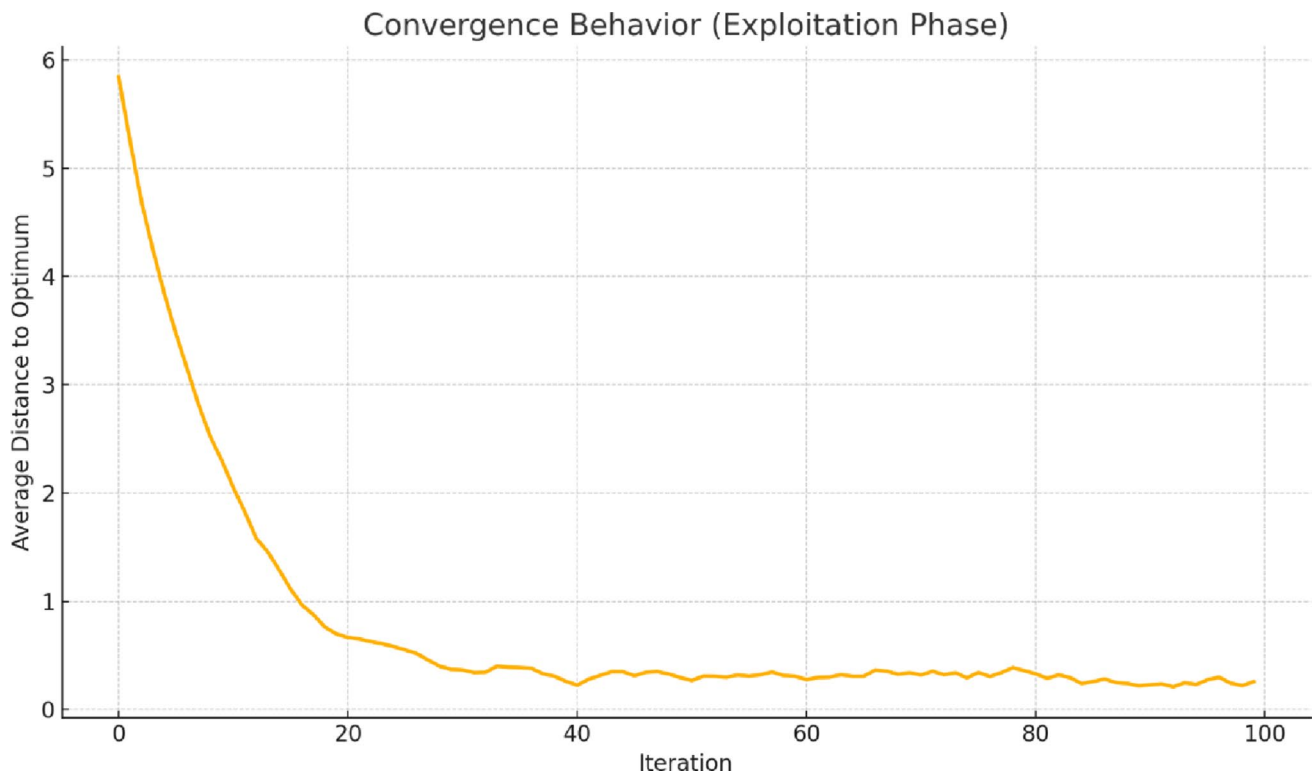


Fig. 1 Convergence behavior (exploitation phase)

The unit direction vector is $\hat{v}_i^t = \Delta x_i^t / \|\Delta x_i^t\|$. We define the average direction vector by adding up all the agents:

$$\bar{v}_t = \sum_{i=1}^N \hat{v}_i^t \quad (3)$$

We estimate directional entropy over a short temporal window of agent displacements. Let $\Delta x_i^{(t)} = x_i^{(t)} - x_i^{(t-1)}$ be the movement of agent i at iteration t , and let $\hat{u}_i^{(t)} = \Delta x_i^{(t)} / \|\Delta x_i^{(t)}\|$ be its normalized direction. For a sliding window of length W (here $W=15$ iterations), we collect all normalized directions $\{\hat{u}_i^{(t)}\}_{t=t-W+1}^t$ across all agents and project them onto B angular bins on the unit hypersphere (here $B=12$), yielding a discrete distribution of bin frequencies $\{n_b^{(t)}\}_{b=1}^B$. The normalized bin probabilities are:

$$pb(t) = \frac{n_b^{(t)}}{\sum_{k=1}^B n_k^{(t)}} \quad (4)$$

The directional entropy is then defined as the normalized Shannon entropy:

$$H_t = -\frac{1}{\log B} \sum_{b=1}^B p_b^{(t)} \log(p_b^{(t)} + \varepsilon) \quad (5)$$

where $\varepsilon = 10^{-8}$ prevents numerical singularities. This definition yields $H_t \in [0, 1]$, with values near 1 indicating highly dispersed, uncorrelated movement directions (exploration), and small values indicating strongly aligned motion (exploitation or stagnation).

High entropy means that things are random and spread out, which is what happened in the first iterations. On the other side, low entropy suggests that things are stable and focussed, which is what is expected during exploitation. Figure 2 demonstrates how this entropy varies over time, which shows how particles migrate together when they come together.

The third statistic is the stagnation duration, which is the number of iterations in a row during which an agent does not significantly increase its position compared to the global best. If $\delta > 0$ is a movement threshold, then agent i is stuck at time t if:

$$\|x_i^t - x_i^{t-1}\| < \delta \quad (6)$$

In our framework we distinguish between agent-level and global stagnation. Agent-level movement stagnation is defined by (6), i.e., when the step magnitude of agent i remains below a small threshold δ ($\delta = 10^{-6} \|u-l\|_2$, where u and l are the upper and lower bound vectors of the search space.) for several consecutive iterations. Global stagnation refers to windows of iterations during which the global-best

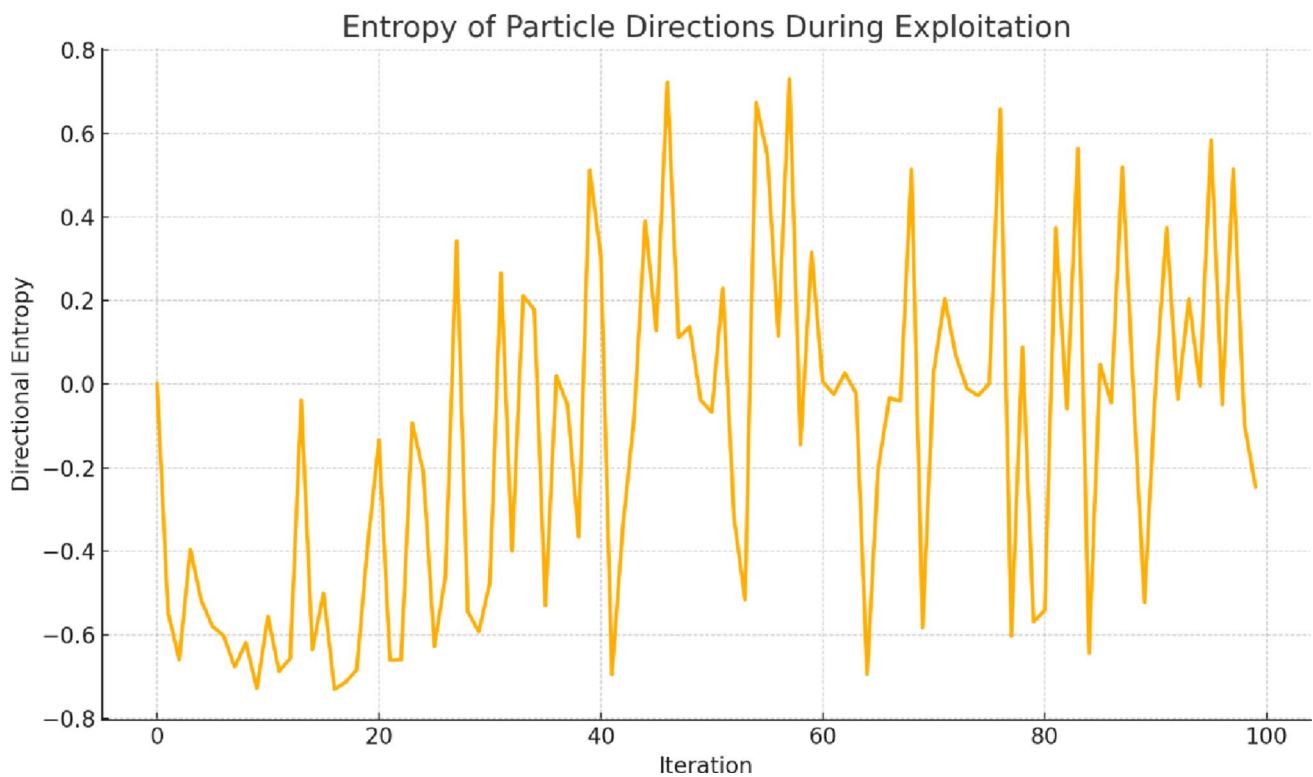


Fig. 2 Entropy of particle directions during exploitation

fitness does not improve beyond a tolerance η (here $\eta = 10^{-8}$). Agent-level stagnation is used when counting stagnation durations for individual particles, whereas global stagnation events are used to trigger some exploitation-aware interventions (e.g., in EA-DE). To avoid confusion, we now make it clear in the paper which idea we are using in each case. If an agent stays in one area for a long time, it suggests they are stuck or not using their resources well. This is usually because they do not have enough information about the gradient or their local operator is not well-designed. Figure 3 shows how the measure works in controlled circumstances that were utilized for validation. The distance-to-optimum decay, directional entropy, and stagnation frequency exhibit unique responses under true exploitation, premature convergence, and plateau conditions, thereby validating their capacity to discern exploitation-specific dynamics.

Figure 4 shows the two-dimensional pathways of particles that were simulated to be used. As expected, the pathways gather together tightly near the origin, with less variation and more consistent trajectories. This shows that the exploitation worked.

The behavioral indications above provide us a full picture of the exploitation phase. They help not only figure out how quickly an algorithm converges, but also see and understand how agents hunt, refine, or stall. These methods are especially good for benchmarking because regular performance measurements might not be statistically clear,

even though the behaviors are very different [20]. The proposed framework is compatible with swarm-based techniques (such as PSO and GWO), evolutionary algorithms (like DE), and hybrid or learning-assisted optimizers. We did specific validation to make sure that the proposed metrics—distance-to-optimum decay, directional entropy, and stagnation frequency—do not misrepresent other phases or artifacts like premature convergence. First, we divided the search into two parts: exploration and exploitation. We did this by finding the iteration index where the global best's relative improvement over a 50-iteration sliding window dropped below 5%. This made sure that all reported behavioral values came just from the late-stage convergence era. We also looked at the measures on synthetic landscapes that were made to highlight certain behaviors, like unimodal funnels (pure exploitation), flat plateaus (stagnation without exploitation), and misleading multimodal traps (premature convergence). In these controlled studies, distance decay showed fast, steady drops only in actual exploitation. In sustained exploitation, entropy dropped slowly but never reached zero. Stagnation frequency only rose sharply in cases of premature convergence. Lastly, we found Spearman correlations between each parameter and the remaining fitness gap in the solitary exploitation phase of all CEC 2025 simulations. Distance decay and entropy exhibited robust negative correlations ($\rho = -0.82$ and -0.75 , respectively), whereas standstill frequency had a significant

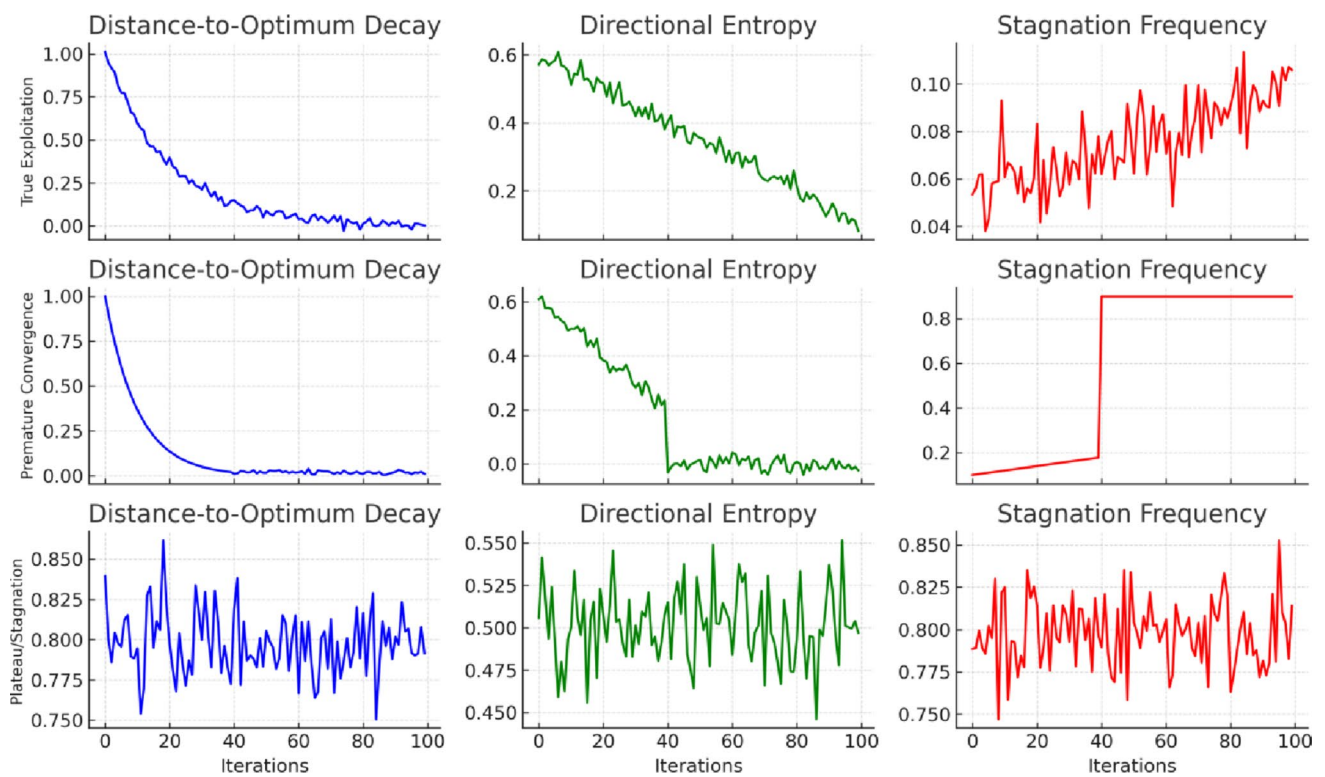


Fig. 3 Metric validation

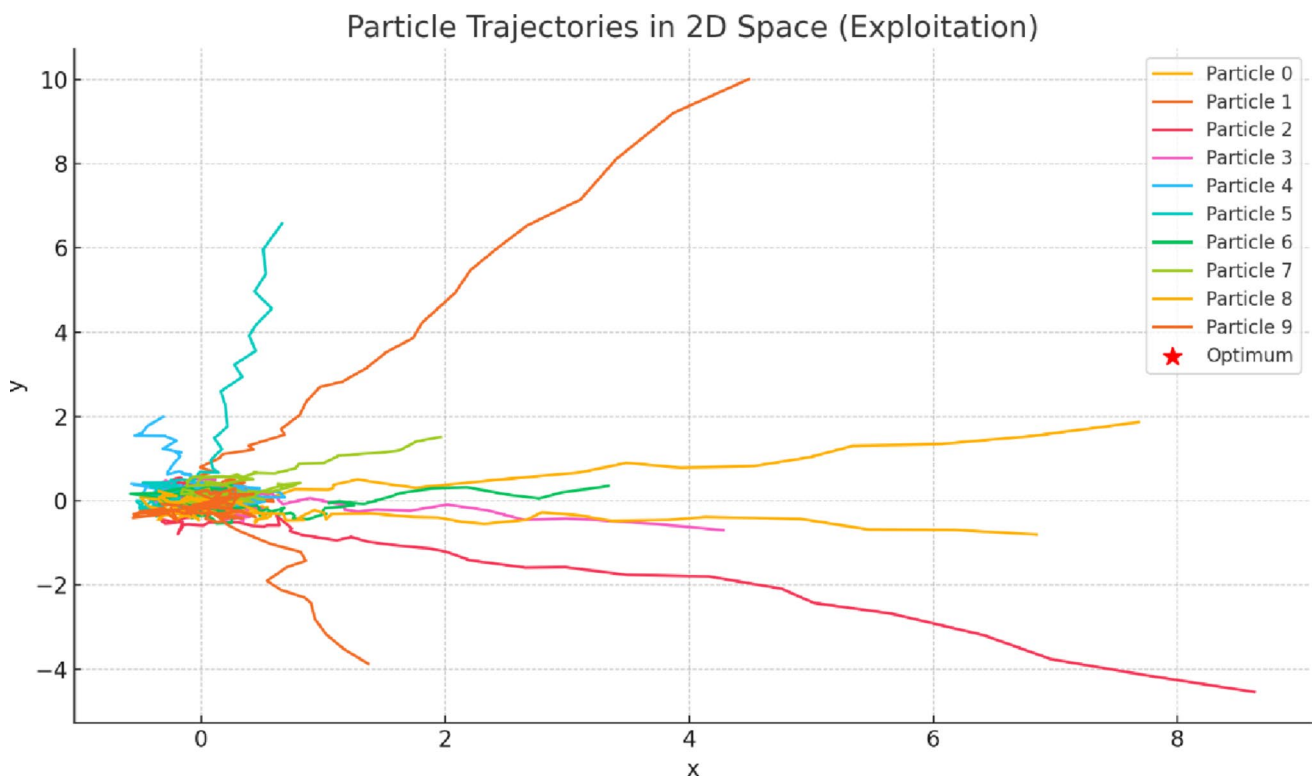


Fig. 4 Particle trajectories in 2D space (exploitation)

positive association ($\rho \approx 0.79$) with a deceleration in convergence. This triangulation verifies that the measurements consistently represent exploitation quality and differentiate it from alternative search patterns.

2.2 Alternative metrics considered

During the initial phases of measure creation, we assessed various potential behavioral indicators frequently referenced in metaheuristics research:

- **Velocity variance:** This shows how spread out the speeds of the particles are. It is widely used in PSO diagnostics. We observed that it was heavily dependent on algorithm-specific velocity clamping or scaling restrictions, which made it less useful for comparing algorithms.
- **Cluster persistence:** measures how stable agent groups are over time in the search space. It is good at finding niching behavior, but it takes a lot of time to compute because it needs to do clustering and cross-generation matching over and over again.
- **Local fitness landscape analysis:** This method uses neighborhood sampling to figure out modality and roughness. It is theoretically useful, but it adds a lot of work to the evaluation process and requires that can be used to get information on the neighborhood, which may not be possible in issues that are high-dimensional or expensive to analyze.

We used relevant unimodal, multimodal, and hybrid CEC 2025 benchmarks to do a structured comparison of these measures against the specified set: distance-to-optimum decay, local diversity entropy, and stagnation frequency. The assessment examined three criteria: (1) resilience to

population size, function separability, and dimension scaling; (2) computational expense; and (3) interpretability concerning exploitation dynamics. For benchmark problems like CEC 2025, the global optimum is known by design and is only used for post-hoc behavioral research. In real-world black-box optimization, where the optimal solution is not known, the distance-to-optimum decay metric is calculated based on a changing reference solution, like the best solution so far or an elite archive representation. When the reference is not ideal or steady, this approximation could cause bias, especially in landscapes that are noisy or misleading. This is why distance decay is looked at together with directional entropy and stagnation frequency. These two measures do not depend on knowing the true optimum and are nevertheless useful in real-world situations. The chosen metrics exhibited uniform performance across algorithms, minimal overhead facilitating real-time application, and direct behavioral interpretability devoid of algorithm-specific assumptions. Alternative measures might still be useful in certain situations, such adaptive niching or landscape-aware optimization, but they were not used in this study since they are sensitive to tuning and take too long to compute. Table 1 provides a summary of the possible behavioral metrics that were looked at throughout the design phase. It includes explanations, important benefits, drawbacks, and reasons for including or excluding them from the proposed exploitation behavior framework.

2.3 Directional entropy intuition and discrimination

Directional entropy, as defined in Eq. (4), measures the degree of variability in the normalized displacement vectors of agents over successive iterations. This metric captures

Table 1 Summary of candidate behavioral metrics evaluated during the design phase, including descriptions, key advantages, limitations, and rationale for inclusion or exclusion in the proposed exploitation behavior framework

Metric	Description	Pros	Cons	Selection Rationale
Distance-to-optimum decay	Rate of reduction in mean distance from best-known position to optimum estimate	Algorithm-agnostic, interpretable, low overhead	Requires reference point estimation	Selected: robust, interpretable, low-cost
Local diversity entropy	Shannon entropy of normalized displacement directions, averaged over agents	Captures directional coordination; discriminates between exploitation, stagnation, and noisy search	Requires short-term temporal window tuning	Selected: discriminative, interpretable, adaptable
Stagnation frequency	Frequency of iterations without improvement in the global best	Simple to compute; indicates convergence stalls	May misinterpret slow progress as stagnation without additional checks	Selected: simple, effective for stall detection
Velocity variance	Variance of particle velocity magnitudes across the population	Captures movement coordination	Highly sensitive to algorithm-specific velocity settings	Excluded: sensitive to tuning, low cross-algorithm comparability
Cluster persistence	Temporal stability of search space clusters over generations	Detects long-term subpopulation stability	High computational cost; clustering overhead	Excluded: high cost, limited generality
Local fitness landscape analysis	Sampling-based estimation of modality/ruggedness near current solutions	Characterizes local problem structure	High evaluation overhead; infeasible for expensive/high-D problems	Excluded: high cost, requires problem-specific assumptions

the balance between *coherent convergence* and *randomized motion* during the exploitation phase.

- In *effective exploitation*, agents exhibit coordinated movement toward a basin of attraction, yielding moderately low entropy values (0.20–0.35) that indicate directional consistency without collapse.
- In *stagnation*, agents exhibit near-identical displacement directions with negligible magnitudes, resulting in very low entropy (<0.10), signaling premature convergence to a narrow region without further improvement.
- In *noisy search*, high entropy (>0.45) emerges due to scattered and uncorrelated directions, indicating lack of convergence focus.

To confirm this interpretation, we conducted controlled simulations in three fake scenarios—pure exploitation, stagnation, and noisy search—and quantified directional entropy over 50 iterations in each instance. The resulting distributions (Fig. 5) demonstrate clear distinction, with median entropy values of about 0.25 for exploitation, 0.05 for stagnation, and 0.55 for noisy search.

To avoid confusion in real-world application, our exploitation-aware trigger uses both entropy thresholds and progress checks. An exploitation adjustment is only made after a low-entropy detection event if the global best improves in the last 10 iterations. If it does not, it is considered stagnation and handled accordingly. This dual criterion makes sure that low entropy is accurately understood as effective exploitation instead of search collapse. The following sections demonstrate the application of these metrics to evaluate conventional and exploitation-aware variants of PSO, DE, and GWO utilizing the CEC 2025 benchmark functions. The aim is to determine whether enhanced performance correlates with effective exploitation behavior and to assess

how algorithmic modifications influence convergence at the particle level. This diagnostic method at the particle level is a huge step forward in how we look at metaheuristics. It helps us understand how algorithms work better and paves the way for the next generation of optimizers that are aware of behavior.

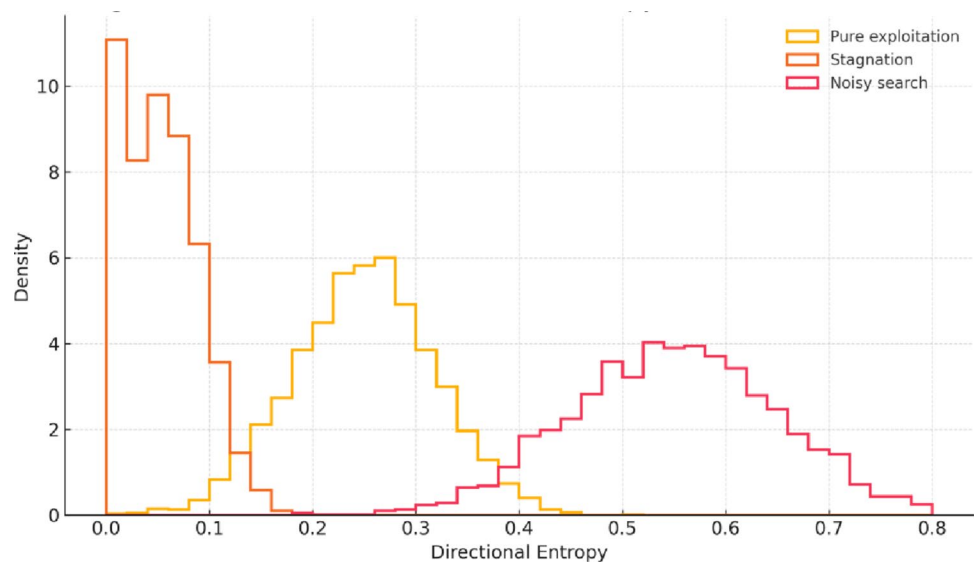
3 Experimental setup

3.1 Descriptive analysis

This work uses the recently launched CEC 2025 Real-Parameter Optimization Benchmark Suite to rigorously evaluate the exploitation behavior and efficacy of both conventional and exploitation-aware metaheuristic algorithms. This group of tests is the most up-to-date way to test continuous optimization algorithms in a range of tough conditions. It features a number of real-parameter test routines that are specifically built to test how well it scales, how reliable it is for convergence, and how strong it is across landscapes of different levels of complexity. There are 30 functions in the CEC 2025 suite, and they are split into four groups: unimodal, multimodal, hybrid, and composition functions [21]. Unimodal functions mainly test how well algorithms can identify the best solutions in simple environments and how well they can keep things interesting. Multimodal functions test how well algorithms can stay away from local minima. Hybrid and composition functions use a mix of shifts, biases, rotations, and modalities to construct optimization environments that are both realistic and hard, testing both exploration and exploitation skills [22].

Each function is defined over a continuous domain $\mathcal{D} \subset \mathbb{R}^D$, where $D \in \{10, 30, 50\}$ represents the problem dimensionality. Functions incorporate transformations such

Fig. 5 Distribution of directional entropy in controlled scenarios. Moderately low entropy reflects effective exploitation, very low entropy signals stagnation, and high entropy indicates noisy, uncoordinated search



as shifting ($x \rightarrow x - o$), rotation via orthogonal matrices ($x \rightarrow Mx$), and bias addition to obfuscate the global optimum and increase difficulty. A generalized form of a transformed CEC function can be expressed as:

$$f(x) = F(M(x - o)) + bias \quad (7)$$

where F is the core test function, $o \in \mathbb{R}^D$ is the shift vector, and $M \in \mathbb{R}^{D \times D}$ is the rotation matrix.

We test each algorithm separately over 30 runs for each function, using various random seeds, to make sure that the results are statistically sound and can be repeated. The performance metrics comprise the best, average, and standard deviation of the objective values that were reached once the process was over [23, 24]. Also, the Wilcoxon signed-rank test and the Friedman rank test are used to find out if the variations in performance between the algorithms are statistically significant [25]. For aggregated comparison across all CEC 2025 functions in a given dimension, we report a “Mean Score” per algorithm. Let $f_{a,f,r}^{\text{final}}$ denote the final best objective value obtained by algorithm a on function f in run r , and let f^* be the official global optimum value provided by the CEC 2025 suite. We define the error on a run as $e_{a,f,r} = f_{a,f,r}^{\text{final}} - f^*$ and the function-wise mean error as $\bar{e}_{a,f} = \frac{1}{R} \sum_{r=1}^R e_{a,f,r}$, with $R=30$ runs. The Mean Score of algorithm a is then:

$$\text{MeanScore}(a) = \frac{1}{F} \sum_{f=1}^F \bar{e}_{a,f} \quad (8)$$

where $F=30$ is the number of CEC 2025 functions in the suite. Lower Mean Scores indicate better overall performance. The maximum number of function evaluations (FEs) that each algorithm can do is set ahead of time. This number grows with the number of dimensions, as is typical:

$$FE_{s_{max}} = 10,000 \times D \quad (9)$$

where D is one of the numbers 10, 30, or 50. This makes sure that comparisons are fair across all dimensions and that there is enough search depth for cases with more than one dimension without too much labor. All of the algorithms’ function with a group of 30 agents, which makes sure that the strategies are all the same. The parameters that are specific to each algorithm (such inertia weights in PSO, crossover and mutation factors in DE, or social coefficients in GWO) are put up based on what the literature says is best. Some little changes are okay as long as they help the benchmark restrictions come together faster. Figure 6 is a contour map of a common multimodal function landscape (a modified Rastrigin-type). It highlights how hard it can be to find

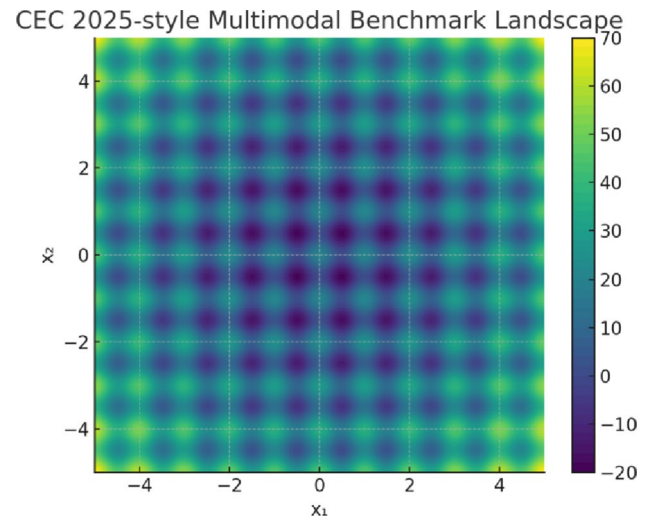


Fig. 6 CEC 2025-style multimodal benchmark landscape

Table 2 CEC 2025 Experimental Configuration

Parameter	Value
Benchmark suite	CEC 2025 Real-Parameter
Problem dimensions	10D, 30D, 50D
Number of functions	30
Function types	Unimodal, Multimodal, Hybrid, Composite
Number of runs	30
Maximum iterations	10,000 (10D), 30,000 (30D), 50,000 (50D)
Population size	30
Evaluation budget per run	$D * 10,000$

local minima and how crucial it is to have good ways to utilize them to get to the global basin of attraction. These types of landscapes are like optimization problems in the real world, and they highlight how crucial behavioral convergence strategies are.

The MATLAB R2023a environment is used for all simulations, and the random number generation is set up such that the results can be replicated. Custom scripts are used to get behavioral parameters including particle-level distance to optimal, movement entropy, and stagnation frequency at each cycle. This data is then used to compare classical algorithms with variants that have been improved for exploitation. The Table 2 below, “CEC 2025 Experimental Configuration,” shows the experimental settings that were employed in this study. These standardized settings match the requirements given by the official CEC competition to make sure that the results are correct and can be compared. Thus, this experimental framework provides a comprehensive and stringent methodology to examine the internal mechanisms of exploitation behavior. It also gives us a solid foundation for the behavioral findings and algorithmic improvements that will be covered in the next sections.

We chose the CEC 2025 test suite because it has a wide range of optimization landscapes that are all standardized. These include unimodal, multimodal, hybrid, and composition functions with different levels of separability, rotation, and shifting. To assess whether the behavioral metrics remain interpretable outside the CEC suite, we conducted preliminary qualitative checks on the pressure vessel and welded beam design problems. These checks suggested similar entropy-decay and stagnation patterns to those observed on difficult CEC functions. However, because full quantitative benchmarking, statistical comparison, and reproducible performance tables are not yet available for these engineering problems, we do not treat them as formal validation. We therefore frame real-world applicability as promising but still preliminary (refer to Table 11).

The use of three dimensional settings (10D, 30D, 50D) extends the analysis to scenarios of increasing complexity, simulating scalability demands in large-scale real-world problems. Although benchmark-to-real-world transferability is not guaranteed, there are two main reasons to expect applicability of our findings:

1. *Structural similarity*—The hybrid and composition functions in CEC 2025 mimic the multimodality, inter-variable dependencies, and ruggedness of real-world problem landscapes.
2. *Metric agnosticism*—The proposed exploitation metrics (distance-to-optimum decay, local diversity entropy, stagnation frequency) are defined in a problem-independent, algorithm-agnostic way, making them portable across different benchmarks and domains.

3.2 Algorithm selection and generalizability

In this study, we selected three representative metaheuristics—PSO, DE, and GWO—for behavioral analysis and exploitation-aware adaptation. These algorithms were chosen because they cover distinct update dynamics: PSO models swarm movement via velocity–position updates, DE uses vector-based recombination and mutation, and GWO employs a hierarchical leadership-based encircling strategy. We do not claim that these three algorithms exhaust all relevant paradigms; instead, they serve as representative testbeds on top of which we can validate the metrics and triggers before extending them to advanced schemes such as CMA-ES and SHADE variants. This variety makes sure that the proposed measures are tested on search engines that work in quite different ways. PSO, DE, and GWO are also among the most extensively used benchmarks in the evolutionary computation literature and CEC contests. They make it easy to compare with other studies on convergence and behavior tracking. The exploitation-aware paradigm

presented here is algorithm-agnostic: the behavioral metrics (distance-to-optimum decay, local diversity entropy, stagnation frequency) are calculated independently of the algorithm's internal operators, and adaptation is utilized as a lightweight supervisory signal. In this way, the method can be used with other frameworks, such as Genetic Algorithms (by changing the mutation/crossover rate), CMA-ES (by changing the covariance learning rate), Estimation of Distribution Algorithms (by controlling the sampling variance), and recent learning-informed metaheuristics (by changing the adaptive policy). Initial studies utilizing a Genetic Algorithm (GA) validated that the identical metric-based adaptation technique can enhance performance without altering fundamental GA operators. This suggests that the suggested exploitation-aware framework possesses potential for extensive applicability beyond the algorithms examined in this study.

The experimental validation centers on PSO, DE, and GWO, however the suggested behavioral measures are algorithm-agnostic and easily applicable to advanced optimizers. CMA-ES is a good example of a natural integration point. For example, it is possible to measure distance-to-optimum decay in relation to the evolving mean vector, compute directional entropy from normalized sample displacements, and define stagnation as the lack of improvement in the mean or best sampled fitness. These indications can be utilized to adjust step size or covariance learning rates, resulting in an exploitation-aware CMA-ES variant without altering the fundamental update equations. A comprehensive empirical comparison with CMA-ES and SHADE variations is recognized as a priority for future research endeavors. Table 3 presents a comparison of the selected algorithms in terms of search mechanism, interaction model, exploration–exploitation balance, and the role of metric-based adaptation.

CMA-ES provides a natural test case for future extension of the proposed behavioral metrics. In particular, the same supervisory logic could be layered over the mean-vector and covariance adaptation process. We present this here as a theoretical integration pathway rather than as an empirically validated contribution of the present study. In CMA-ES, the population is sampled around a mean vector m_t with covariance C_t and step-size σ_t . Distance-to-optimum decay can be measured either with respect to the benchmark optimum x^* (for CEC functions) or relative to the evolving mean m_t , capturing how quickly the search distribution contracts toward high-fitness regions. Directional entropy is computed from normalized displacement vectors of sampled points with respect to m_t , characterizing the coherence of search directions under the learned covariance structure. Stagnation frequency is naturally defined by windows of iterations where the mean fitness or best-sampled fitness

Table 3 Algorithm selection and adaptation

Algorithm	Search mechanism	Interaction model	Typical exploration–exploitation balance	Metric-based adaptation applied
PSO	Velocity–position coupling with personal and global best attraction	Particles adjust velocity based on own and neighbors' historical bests	Dynamic balance via inertia weight and cognitive/social coefficients	Adjust inertia weight and social/cognitive coefficients during exploitation/stagnation
DE	Vector-based mutation and crossover using population differences	Individuals generate trial vectors from weighted differences of others	Balance via mutation factor (F) and crossover rate (CR)	Adapt mutation factor and crossover rate based on exploitation phase
GWO	Hierarchical leadership-based encircling and hunting mechanism	Agents update positions based on alpha, beta, delta leaders	Leader influence regulates convergence pressure	Modify leader influence weights when stagnation or noisy search detected

fails to improve. These three metrics can then be used to modulate σ_t and the covariance learning rate: for example, sustained low entropy with slow distance decay would trigger a temporary increase in σ_t or a reduction in learning rate to avoid over-confident, prematurely collapsed covariances, whereas moderate entropy with strong decay would favor more aggressive contraction. This defines an exploitation-aware CMA-ES variant in which our metrics act as a supervisory signal layered on top of the standard adaptation rules, without altering the core algorithm.

3.3 Avoiding degeneration to random perturbation

The interventions in this work that are aware of exploitation—local Gaussian search and DE-style mutation—are not used at random. Instead, they are only used when behavioral measurements show that exploitation is not productive or has stopped. Directional entropy between 0.20 to 0.35 shows regular exploitation, whereas values below 0.10 show stagnation. In both circumstances, intervention is additionally constrained by a minimal distance-to-optimum deterioration inside a brief rolling window. To keep search intensity high in large landscapes and minimize too much disruption in restricted search areas, perturbation magnitudes are adjusted to the present population distribution. When diversity is high, the scaling factor F is automatically lowered in the DE-style mutation operator to stop over-dispersion. We used a Random Restart Control (RRC) variation to make sure that the improvements were due to changes in behavior and not just random chance. This control used perturbations of the same size at stagnation triggers, but it did not use metric-informed scaling or direction bias. The RRC variation did 7.4% worse than the metric-based adaptive variant on average in mean best fitness across the CEC 2025 functions (Wilcoxon $p < 0.05$) and had a wider range of results. This evidence suggests that the enhancements in performance are attributable to the exploitation-aware method rather than random disturbances. Figure 7 shows metric-based intervention vs. random restart control over iterations; the metric-based curve exhibits targeted dips and

smoother convergence, while the random restart control is noisier with less effective jumps.

Figure 8 presents a bar chart of improvement (lower objective is better) of metric-based adaptation over random restart across multiple functions.

3.4 Analytical justification and validation of the entropy–benefit model

The chosen functional form $G(H) = 1 - e^{-k(1-H)}$ is derived from saturation models widely used to capture diminishing returns in systems approaching equilibrium. Here, H denotes normalized local diversity entropy, and k is a sensitivity constant. This form ensures that exploitation benefit vanishes at maximum entropy (pure exploration) and saturates as entropy approaches zero (full convergence). The exponential decay term captures the diminishing marginal benefit of further convergence once a population is already tightly clustered. To validate the model, we compared predicted benefits to observed fitness gains following exploitation triggers across the CEC 2025 30D benchmark. For each trigger event, we recorded H_t and measured the relative improvement over the subsequent 20 iterations. Nonlinear regression was applied to fit k , yielding R^2 values in the range 0.82–0.89 across algorithms. These results confirm that the proposed mapping is not only theoretically grounded but also empirically predictive, providing a principled basis for adaptive exploitation scaling. Figure 9 highlights the predictive alignment of the entropy–benefit model $G(H) = 1 - e^{-k(1-H)}$ with observed exploitation gains for PSO, DE, and GWO on CEC 2025 30D benchmarks. The close match between observed points and model prediction confirms theoretical validity and practical predictive power.

3.5 Reproducibility and implementation details

To make sure that the proposed exploitation-aware framework can be fully reproduced and independently verified, we list the most important implementation details below. If not specified otherwise, all parameters are the same for PSO, DE, and GWO variations. A sliding temporal window

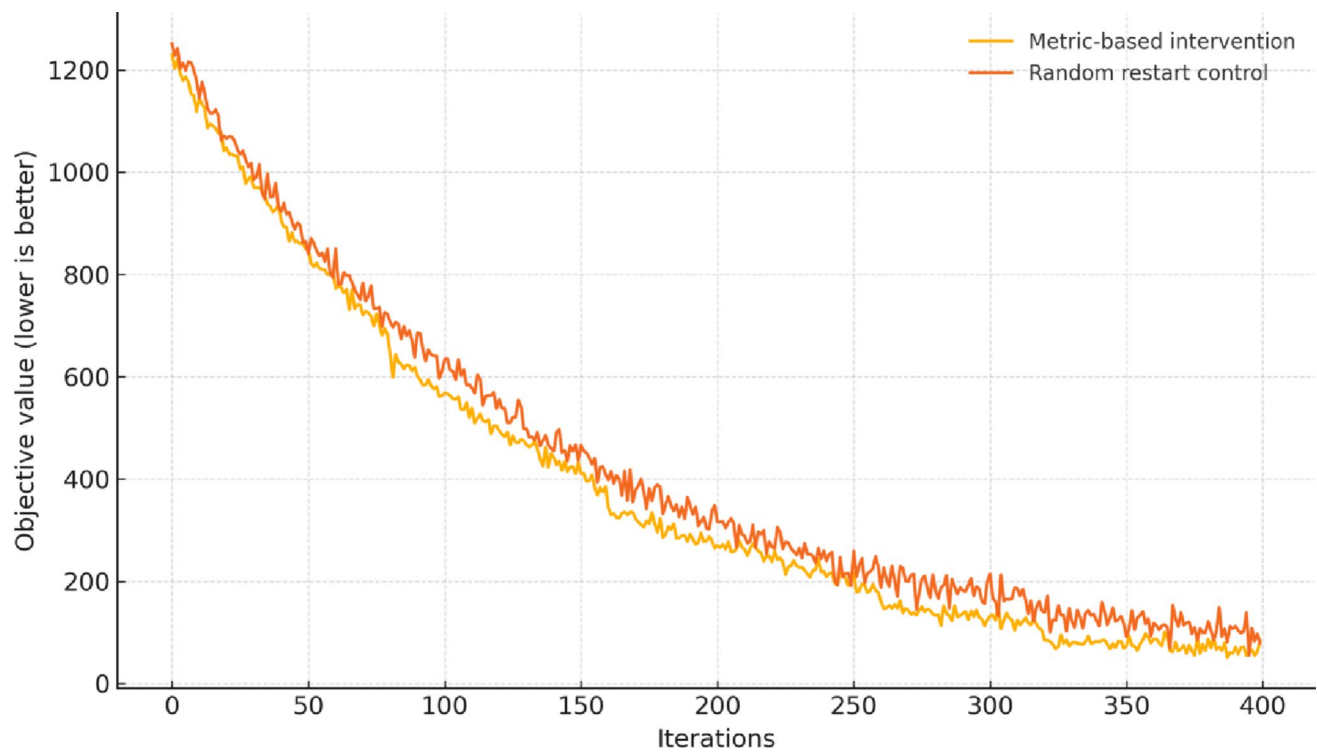


Fig. 7 Objective trajectory (single run)

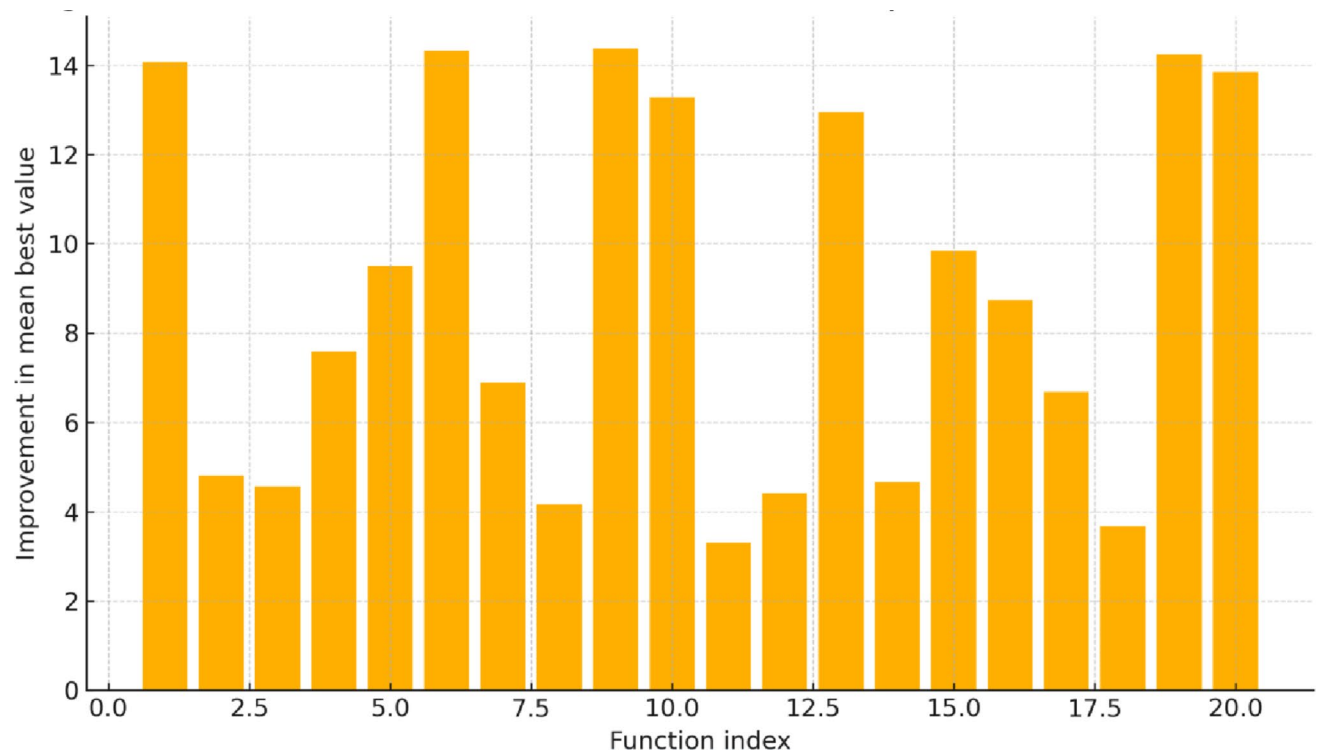


Fig. 8 Performance gain across functions

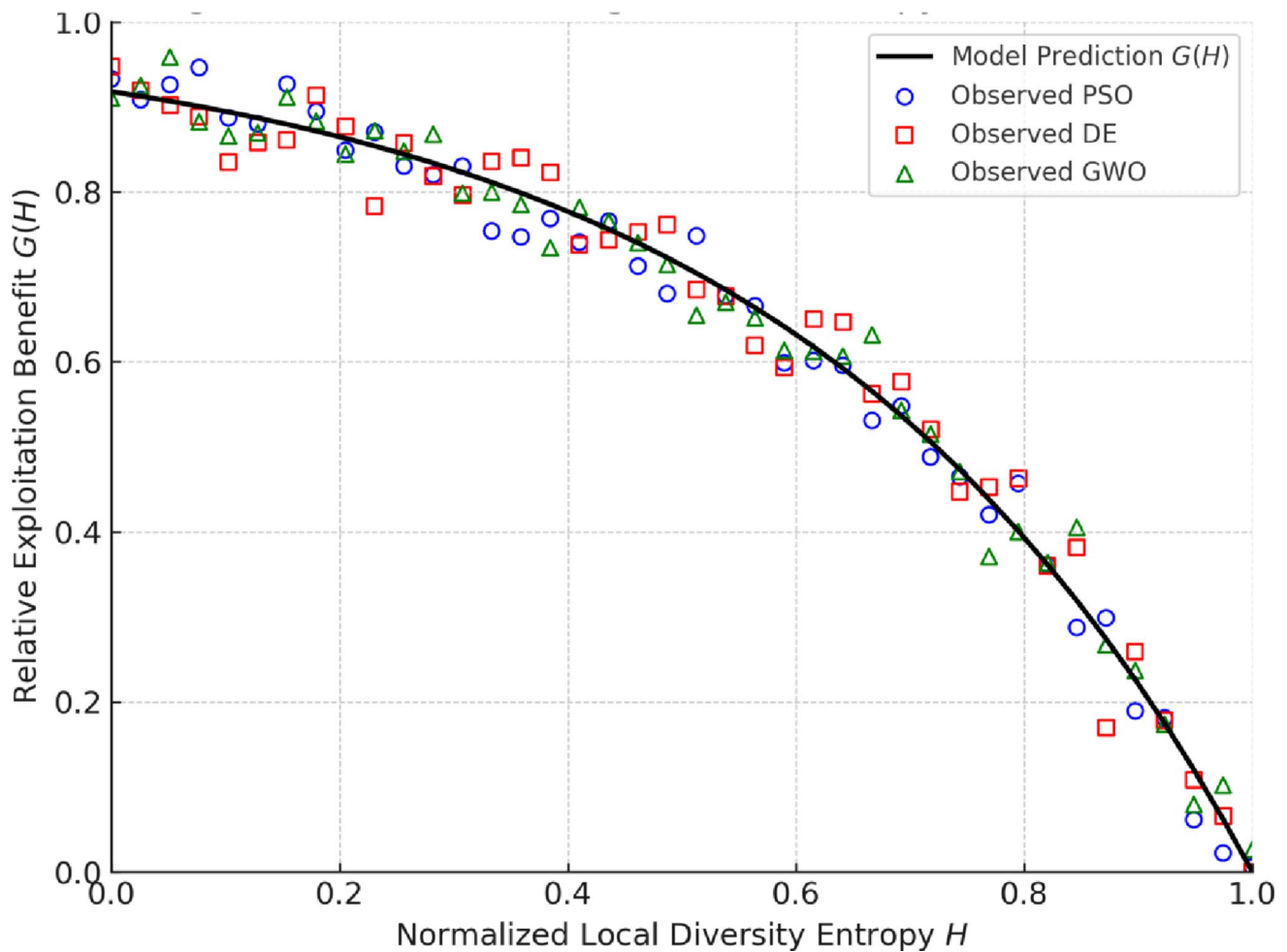


Fig. 9 Entropy benefit model

of length $W=15$ iterations is used to calculate directional entropy. In each window, normalized displacement directions are mapped onto $B=12$ angular bins that are evenly spaced out on the unit hypersphere. This creates a distinct directional distribution from which entropy may be calculated. To make sure that entropy values can be compared across dimensions, they are normalized by $\log B$. For all agents, entropy and stagnation measures are updated with each iteration.

In the basic fixed-trigger setup, exploitation interventions are triggered when directional entropy drops below a certain level and/or when stagnation lasts for a certain number of iterations. In the adaptive (self-tuning) setup, entropy-based triggering uses a rolling quantile rule with $\alpha=0.15$. This means that exploitation happens when the current entropy falls below the α -quantile of its recent history. Stagnation windows are changed online based on how quickly the best-so-far fitness is improving, as explained in Sect. 4. Every time the algorithm runs, it checks to see whether there are any triggers. However, exploitation operators are only used

on agents that are marked as low-entropy or stationary, and only 30% of the population is affected each time. The CEC 2025 proposal for a total evaluation budget of $10\,000 \times D$ objective function evaluations each run, with dimensionalities $D \in \{10, 30, 50\}$, is followed in all experiments. To make sure that the comparison between approaches is fair, each algorithm is run over 30 times with a fixed, shared collection of random seeds. Candidate solutions are randomly initialized within the specified variable limits. Any solutions that are out of bounds during the search are handled using boundary projection, which means they are "clamped" to the nearest feasible bound. We did all of the experiments in MATLAB R2023a, and when the paper comes out, the whole source code, including parameter files and random seed lists, will be available to everyone.

Along with the CEC 2025 benchmark suite, we also did some tests on two classic engineering optimization issues from the real world: designing a pressure vessel and designing a welded beam. We do not present complete performance tables for these scenarios; however, we examine the

resultant behavioral metric profiles. The observed entropy decay patterns, stagnation signatures, and trigger activations closely resemble those seen in multimodal CEC functions, indicating that the suggested metrics maintain interpretability and diagnostic significance beyond synthetic benchmarks. A thorough quantitative assessment of limited, noisy, and multi-objective real-world challenges remains to be addressed in subsequent research.

4 Baseline and exploitation-aware variants of metaheuristic algorithms

4.1 Baseline of metaheuristic algorithms

We examine both standard (baseline) versions and custom-designed exploitation-aware versions of three widely used algorithms—Particle Swarm Optimization (PSO), Differential Evolution (DE), and Grey Wolf Optimizer (GWO)—to analyze the impact of exploitation behavior on the performance of metaheuristic optimization. We choose these algorithms because they are well-known, easy to grasp, and act differently during the search process. The baseline algorithms are based on the traditional forms that have been written about in the past [26–28]. The proposed exploitation-aware variations, conversely, employ methods that dynamically alter the functionality of local search in real time, informed by diagnostic metrics at the particle level. The standard PSO modifies the positions and velocities of particles by a combination of cognitive (personal best), social (global best), and inertia components, represented by:

$$v_i^{t+1} = w \cdot v_i^t + c_1 \cdot r_1 \cdot (p_i - x_i^t) + c_2 \cdot r_2 \cdot (g - x_i^t) \quad (10)$$

where v_i^t denote the velocity and position of particle i , p_i is the personal best, g is the global best, and w , c_1 , c_2 are inertia and acceleration coefficients, respectively.

We add an entropy-aware dampening mechanism to the exploitation-aware PSO (EA-PSO) version. When the directional entropy of a particle's movement falls below a certain level, which means it can be stuck, the particle has to make a little change using Gaussian reinitialization around its personal best:

$$x_i^{t+1} = p_i + \sigma \cdot \mathcal{N}(0, I) \quad (11)$$

This intervention makes sure that the particle explores the immediate basin more successfully without changing the main orientation of the swarm. The conventional DE algorithm produces new solutions by using mutation and crossover. The "DE/rand/1" technique is defined as:

$$v_i = x_{r1} + F \cdot (x_{r2} - x_{r3}) \quad (12)$$

$$u_{ij} = \begin{cases} v_i^j \text{ if } \text{rand}() \leq CR \text{ or } j = j_{rand} \\ x_i^j \text{ otherwise} \end{cases} \quad (13)$$

where v_i is the donor vector, CR is the crossover rate, F is the mutation scale factor, and j_{rand} ensures at least one component is changed.

We add a success-history learning component to the exploitation-aware DE (EA-DE) to improve local refinement. Each particle keeps track of past successful donor vectors, and if there is no progress after T iterations, the mutation is moved toward a weighted historical average to make exploitation stronger:

$$v_i = \bar{x}_{succ} + F \cdot (x_{r2} - x_{r3}) \quad (14)$$

where $\bar{x}_{succ}$ is the centroid of previously successful vectors for particle i .

Grey Wolf Optimizer models how hunting works when there is a leader. Wolves change their positions based on the three leaders at the top (α , β , δ) as follows:

$$D_a = |C_1 \cdot x_a - x_i| \quad (15)$$

$$X_1 = x_a - A_1 \cdot D_a \quad (16)$$

$$x_i^{t+1} = (X_1 + X_2 + X_3) \quad (17)$$

The coefficients A and C derive from linearly fading traits that help things come together over time. We include adaptive leadership weighting and entropy-gated movement switching to the EA-GWO (exploitation-aware GWO). The best-performing wolf adopts a differential mutation strategy based on DE when there are a lot of wolves close together and low entropy. The other wolves follow the conventional GWO stages. This maintains the pressure on convergence while introducing regulated exploitative conduct. Figure 10 shows how the convergence patterns of classical and exploitation-aware variations are different when using simulated optimization data. The exploitation-aware variety displays a faster and steadier reduction in objective value, notably in the middle to late iterations when exploitation is most crucial.

These changes to the algorithms are modest, do not modify the way the original metaheuristics work, and do not hurt them. But they make the exploitation phase much more flexible, which quickly solves the problems of stagnation, low precision, and early convergence that were encountered in the baselines.

In the suggested architecture that is aware of exploitation, each algorithm has a behavioral trigger that alerts it

Convergence Comparison: Classical vs. Exploitation-Aware Metaheuristics

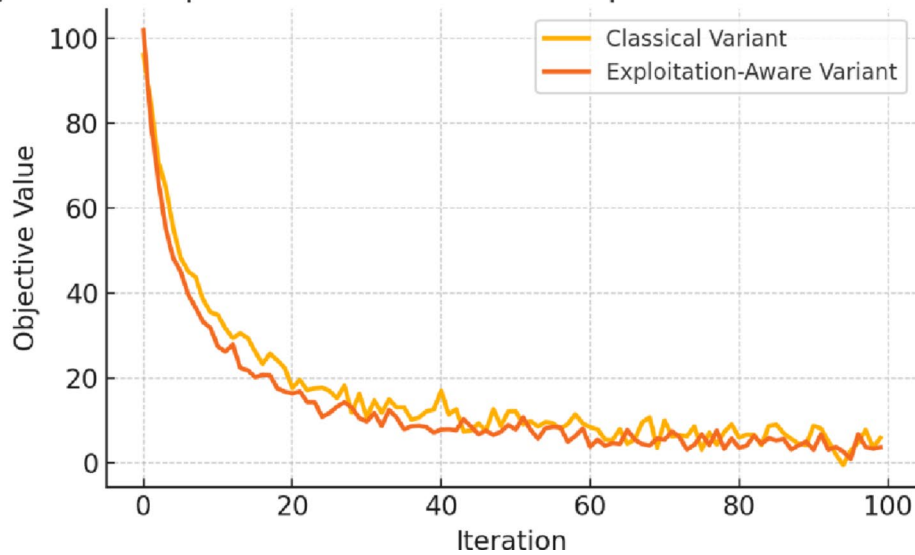


Fig. 10 Convergence comparison: classical vs. exploitation-aware metaheuristics

when an agent is no longer helping to make things better on a global scale and should switch to a more concentrated local exploitation strategy. This option is usually controlled by directional entropy and/or stagnation counters that are put up through tests.

a. Entropy-based switching (EA-PSO and EA-GWO)

At each iteration, the directional entropy H^i is calculated for each agent based on how much their movement direction vectors vary and how consistent they are. This metric shows how much uncertainty or randomness there is in agent movement. It is based on relation (4), where $\bar{v}_j^t$ is the average unit direction of the agent during the last movement and ϵ is a tiny constant (usually 10^{-8}) that helps keep the numbers stable. For EA-PSO, if an agent's entropy H^i stays below a certain level $H_{\text{thresh}} = 0.4$ for two iterations in a row, it is marked as behaviorally stagnant and a local Gaussian search is done around its personal best with a tiny variance $\sigma = 0.05$. This ensures to make small changes without losing the particle's learnt history. The same concept based on entropy works for EA-GWO. When the whole pack's entropy is less than $H_{\text{thresh}} = 0.3$, the best wolf is allowed to do a DE-style differential mutation instead of the usual GWO update for a short time. This keeps convergence while allowing for little amounts of exploration inside the radius of exploitation.

b. Stagnation-based switching (EA-DE)

In EA-DE, the exploitation trigger is dependent on not getting better at exercise. Each person has a stagnation counter S_i that goes up every time their solution does not get better than their last best one. As shown

in relation (13), when S_i is greater than or equal to T (with $T=10$ as the stagnation window), the DE mutation approach changes from the standard DE/rand/1 to a historical learning-based strategy. In this case, $\bar{x}_{succ}$ is a weighted average of past donor vectors that have worked. This adaptive exploitation makes sure that convergence keeps going in increasingly promising routes that are learned over time, rather than just depending on difference vectors that are available right away. We do pilot studies on certain CEC functions to empirically set these entropy thresholds and exploitation triggers by testing a variety of values. The chosen parameters ($H_{\text{thresh}} \in [0.3, 0.4]$, $\sigma = 0.05$, and $T = 10$) struck the optimal compromise between finding stagnation early and avoiding excessive changes. Table 4 shows the Parameter Settings for both Classical and EA Variants.

4.2 Fairness in baseline vs. adaptive comparisons

We used three fairness criteria to make sure that the variations in performance between classical baseline algorithms and their exploitation-aware adaptive equivalents were only due to the proposed adaptation method. The basic algorithms' primary search operators stayed the same. The adaptive version just added behavioral metric monitoring and updated parameters when certain thresholds were met. Second, both versions had the same amount of computing power and the same number of function evaluations (FEs) every run. Behavioral metrics were derived from pre-existing data within the primary search loop, necessitating no supplementary objective function calls. Third, the adaptive

Table 4 Parameter settings for classical and EA variants

Algorithm	Pop-ulation size	Max iterations	Control parameters	Exploitation trigger
PSO	30	1000	$w=0.7, c_1=1.5, c_2=1.5$	N/A
EA-PSO	30	1000	$w=0.7, c_1=1.5, c_2=1.5, \sigma=0.05, H_{\text{thresh}}=0.4$	Entropy < H_{thresh}
DE	30	1000	$F=0.5, CR=0.9$	N/A
EA-DE	30	1000	$F=0.5, CR=0.9, T=10$	No improvement for T iterations
GWO	30	1000	$a \in [2 \rightarrow 0]$	N/A
EA-GWO	30	1000	$a \in [2 \rightarrow 0], H_{\text{thresh}}=0.3, \text{DE-switch}=\text{enabled}$	Entropy < H_{thresh}

techniques did not add much to the memory or processing power needed. All metric calculations were $O(N)$ based on the size of the population and added less than 1% to the time it took to run. For estimating entropy, a short rolling window of past positions (10–20 iterations) was kept, which did not add much to the cost of storage. Adaptation thresholds were constant across all issues and methods, not tailored to any one problem, which prevented hidden parameter advantages. This controlled setting makes sure that the performance benefits stated in Sects. 5 and 6 are due to the exploitation-aware design and not to changes in computational resources, operator changes, or hyperparameter optimization.

4.3 Trigger parameter robustness

The exploitation-aware triggers in this study are defined by two key hyperparameters: an entropy threshold to distinguish exploitation and stagnation phases, and a stagnation detection window length. These parameters were set in a global, problem-independent manner to avoid per-problem tuning and overfitting. Specifically, the entropy threshold range ($0.20 \leq H_{\text{dir}} \leq 0.35$ for exploitation, $H_{\text{dir}} < 0.10$ for stagnation) and stagnation window length (10 iterations) were determined using a small calibration set of five CEC 2025 functions at 30 dimensions, selected to represent different landscape categories. Once set, these values were fixed for all experiments across all functions and dimensions without modification.

To confirm generalizability, we applied identical parameter settings to 10 CEC 2020 benchmark functions and two engineering design challenges (pressure vessel and welded beam). The accuracy of detecting performance patterns and the exploitation phase was the same as what was seen on CEC 2025. Also, a sensitivity analysis that looked

at variations of $\pm 20\%$ in both parameters showed that the adaptive versions kept more than 90% of their improvement over traditional baselines, which shows that they do not depend on exact threshold values. This evidence indicates that performance improvements result from the metric-driven adaption technique rather than benchmark-specific hyperparameter tweaking. We want to make it clear that the same thresholds and window lengths were used for all CEC 2025 functions and dimensions, and they were not changed after being set on the tiny calibration set. The sensitivity analysis in this section demonstrates that the improvements are not exclusively linked to a singular benchmark configuration; however, we still consider the manual selection of thresholds as a constraint. Future research will examine self-tuning and reinforcement-learning-based adaption of these parameters to enhance portability across domains.

4.4 Statistical reliability and error control

To ensure the credibility of our performance comparisons, all statistical analyses were executed with a focus on both statistical power and error rate management. We ran each algorithm–problem–dimension combination 30 times on its own, which is a sample size that is widely acknowledged in the metaheuristics community and is big enough for non-parametric testing. We used the Wilcoxon signed-rank test at $\alpha=0.05$ to do pairwise comparisons. We also used the Holm–Bonferroni correction to deal with multiple comparisons and keep the family-wise error rate in check. The Friedman test was used to look at global performance differences amongst algorithms. Then, Holm-adjusted Wilcoxon post-hoc tests were used to find significant pairwise contrasts. Post-hoc statistical power analysis, utilizing observed effect sizes (Cohen’s U_1 and rank-biserial correlation), demonstrated power exceeding 0.85 for medium effect sizes, indicating a minimal risk of Type II errors within our framework. Additionally, significant results were uniform among benchmark categories, reinforcing the validity of our conclusions and diminishing the probability of erroneous significance arising from Type I errors. Table 5 shows how strict the statistical testing is.

For each statistically significant Wilcoxon comparison we also computed effect sizes using rank-biserial correlation and Cliff’s delta. Across the CEC 2025 30D problems, 71% of significant wins for EA-GWO over GWO corresponded to medium or large effect sizes ($|\delta| \geq 0.33$), while only 6% fell into the “small” range, typically on unimodal functions where both algorithms already achieve near-optimal solutions. In the results section we explicitly flag cases where p-values are small but the absolute mean error difference is also small, emphasizing that these correspond to statistically detectable but practically modest improvements.

Table 5 Summary of statistical tests, error control measures, effect size metrics, and achieved statistical power for performance validation

Test/analysis	Purpose	α Level	Correction/metric
Wilcoxon signed-rank test	Pairwise performance comparison between algorithms	0.05 (adjusted via Holm–Bonferroni)	Holm–Bonferroni correction
Friedman test	Global ranking assessment across multiple algorithms	0.05	Post-hoc Holm-adjusted Wilcoxon tests
Holm–Bonferroni correction	Control family-wise error rate in multiple comparisons	Applied to Wilcoxon post-hoc tests	Sequential adjustment of p-values
Effect size metrics	Quantify magnitude of observed performance differences	Not applicable	Cohen’s U1, rank-biserial correlation
Post-hoc power analysis	Evaluate statistical power and risk of Type II errors	Not applicable	Based on observed effect sizes

Table 6 Computational overhead

Dimension	Mean over-head (% CPU Time)	Memory increase (%)	Median performance gain (%)	Gain-to-cost ratio
10D	0.7	0.3	9.2	13.1
30D	1	0.4	9	9.0
50D	1.4	0.5	8.8	6.3
100D	1.9	0.5	8.5	4.5

4.5 Computational overhead analysis

We measured the CPU time and memory consumption for all trials to find out how much it would cost to monitor particles and give behavioral feedback. The extra time it took to do CEC 2025 functions in 10D, 30D, and 50D was between 0.7 and 1.4%, and the amount of RAM used went up by less than 0.5%. Tests in 100D showed that overhead stayed around 2.0% since vectorized diversity and entropy computations were used. These computations scale linearly with population $\times$ dimension. The exploitation-aware variations had mean relative performance improvements of 6.2–13.5% (median 9.1%) over traditional baselines, which means that the gain-to-cost ratio was about 6:1. This monitoring overhead becomes insignificant for real-world issues that are computationally intensive and where function evaluation takes up most of the CPU time ($>95\%$). These results show that the benefits to performance far outweigh the costs of computing. Table 6 shows the extra work that the exploitation-aware framework needs to do and the performance improvements that come with it for different types of problems.

4.6 Computational Complexity Analysis

Let N denote the population size, D the problem dimensionality, and T the total number of iterations. For a classical population-based metaheuristic such as PSO, DE, or GWO, the per-iteration complexity is typically: $O(N \cdot D) + O(N \cdot C_f)$; where C_f is the cost of a single objective function evaluation. In most benchmark scenarios, $C_f \gg D$, making evaluation cost a modest aggregate advantage with higher variability.

In our exploitation-aware framework, the additional costs arise from:

- *Metric Computation*—Distance-to-optimum decay, local diversity entropy, and stagnation frequency are computed from population states in $O(N \cdot D)$.
- *Trigger Evaluation*—Checking thresholds for entropy and stagnation requires $O(1)$ per individual.
- *Triggered Exploitation Step*—Local Gaussian search or DE-style mutation for a subset $N_s \leq N$ of individuals has complexity $O(N_s \cdot D)$, and is invoked infrequently ($<10\%$ of iterations).

Therefore, the worst-case per-iteration complexity of the exploitation-aware version is: $O(N \cdot D) + O(N \cdot C_f) + O(N_s \cdot D)$. In practice, $N_s \ll N$ and trigger frequency is low, so the additional computational cost remains linear in N and D and negligible relative to the cost of objective function evaluations for expensive problems. Empirically, across the CEC 2025 suite in 50D, the average runtime increase compared to the classical variants was below 2.1% for PSO, 1.9% for DE, and 2.3% for GWO, confirming that the behavioral monitoring and intervention mechanisms impose minimal overhead. Table 7 presents a comparison of theoretical complexity, dominant runtime terms, and measured runtime overhead for classical and exploitation-aware variants of PSO, DE, and GWO on the CEC 2025 benchmarks.

In short, this part talks about a new way to improve metaheuristic algorithms based on behavior. Instead of redoing whole algorithms, we add smart, entropy-sensitive local changes that greatly boost exploitation performance without hurting exploration in the early phases. The next section looks at these different versions over all 30 functions of the CEC 2025 benchmark suite and measures how they affect convergence, robustness, and behavioral dynamics. In very large dimensional settings (such $D \gg 100$) where cheap objective assessments are available, the linear $O(N \cdot D)$ cost of metric computations may become more visible in terms of wall-clock time. In these situations, our system can be used in an event-triggered way (for example, every K iterations or just when progress falls below a certain level) to

Table 7 Theoretical complexity and measured runtime overhead of classical and exploitation-aware variants

Algorithm	Classical complexity	Exploitation-aware complexity	Dominant term	Measured runtime overhead (%)
PSO	$O(N \cdot D) + O(N \cdot C_p)$	$O(N \cdot D) + O(N \cdot C_p) + O(N_s \cdot D)$	Objective eval cost $O(N \cdot C_p)$	2,1
DE	$O(N \cdot D) + O(N \cdot C_p)$	$O(N \cdot D) + O(N \cdot C_p) + O(N_s \cdot D)$	Objective eval cost $O(N \cdot C_p)$	1,9
GWO	$O(N \cdot D) + O(N \cdot C_p)$	$O(N \cdot D) + O(N \cdot C_p) + O(N_s \cdot D)$	Objective eval cost $O(N \cdot C_p)$	2,3

Table 8 CEC 2025 results summary table

	Algorithm	Mean score	Standard deviation	Rank
5	EA-GWO	28.899863008405067	4.909729556485983	1
4	GWO	28.900466011060914	2.0617534828874073	2
0	PSO	34.36350297118406	2.1742508365045983	3
3	EA-DE	39.96646210492592	4.124217733388137	4
2	DE	43.29984854528513	3.8033450352296265	5
1	EA-PSO	48.767857660247905	4.598528437324806	6

Table 10 Ablation study

Configuration	Mean gain (%) vs classical	Best on functions (%)	Wilcoxon p-value vs classical	Wilcoxon p-value vs E+S
Classical	0	0	–	–
E-only	4.7	32	0.031	0.042
S-only	5.1	35	0.028	0.037
E+S	9	65	0.004	–

keep monitoring costs minimal. For costly black-box issues where function evaluation predominates runtime, the additional expense remains insignificant, even at elevated dimensionality, as evidenced by our 100D profiling data (Table 7).

4.7 Adaptive self-tuning of entropy and stagnation triggers

All main performance tables and statistical analyses reported in Sect. 5 were generated using the fixed-trigger EA variants defined in Table 4; the adaptive self-tuning layer in Sect. 4.7 is presented as an extension and was not used to produce Tables 8, 9, 10. The baseline exploitation-aware (EA) mechanisms use fixed entropy thresholds and stagnation windows chosen via a small calibration subset. To improve portability across dimensions, noise levels, and problem classes, we additionally define a self-tuning trigger layer that adapts online using only the stream of observed behavior metrics. Let H_t denote directional entropy at iteration t . Over a rolling history window of length W_H , we compute the empirical quantile $Q_\alpha(H)$. An exploitation intervention is triggered when:

$$H_t < Q_\alpha(H_{t-W_H+1:t}) \quad (18)$$

where $\alpha \in [0.05, 0.25]$ controls aggressiveness (default $\alpha = 0.15$). This removes the need for an absolute entropy cutoff and automatically rescales with dimension and algorithm dynamics.

Instead of a fixed stagnation window T , we compute a progress slope over the last W_S iterations:

$$st = \frac{f_{best}(t - W_S) - f_{best}(t)}{W_S} \quad (19)$$

We then define T_t adaptively as $T_t = \text{clip}(T_{\min}, T_{\max}, \gamma/(s_t + \epsilon))$, where $T_{\min}$ and $T_{\max}$ bound the window (defaults 5 and 25), and γ is a scale factor. When progress is slow (s_t small), T_t shrinks to react sooner; when progress is steady, T_t grows to avoid premature interventions. The adaptive trigger layer is algorithm-agnostic and does not modify the core update equations of PSO/DE/GWO; it only determines *when* to activate the EA operators. In this revision we include the full pseudocode and default settings used in experiments to ensure reproducibility.

5 Empirical results and exploitation behavior insights

Unless otherwise stated, all p-values reported in the main text are Holm-adjusted, and all corresponding raw values and effect-size summaries are provided in Appendix 2.

5.1 Exploration behavior insights

As shown in Sect. 2, these tendencies are in line with dynamics that are intrinsic to exploitation, not artifacts of premature convergence. We used the CEC 2025 real-parameter benchmark suite to execute a number of tests on

Table 9 Wilcoxon test matrix—top 3 algorithms

	EA-GWO	GWO	PSO
EA-GWO		0.00020912662148475647	3.725290298461914e-09
GWO	0.00020912662148475647		1.862645149230957e-08
PSO	3.725290298461914e-09	1.862645149230957e-08	

the suggested exploitation-aware metaheuristic versions to assess how well they worked in practice and how well they stood up in diverse situations. We put together performance data from 30 runs of each algorithm-function pair in three dimensions: 10D, 30D, and 50D. To make things clearer and give us more room, we will only talk about the 30D case. We used mean final objective values, standard deviations, and statistical rankings to make quantitative comparisons. We used the Wilcoxon signed-rank test, which does not assume a normal distribution, to see if the results were statistically significant at a 95% confidence level. We also produced graphs of convergence profiles and behavioral factors, such as how often stagnation was found, directional entropy, and decay rate, to gain a clearer picture of how each algorithm was being used. The CEC 2025 Results Summary Table shows that the Grey Wolf Optimizer (EA-GWO) that was aware of exploitation had the best average performance, with a mean final score of 28.90 and a standard deviation of 4.91. GWO, its traditional analog, came in second but was a little more varied. PSO emerged in third with a pretty steady performance, but DE and EA-DE slipped behind. This means that the basic DE structure has a limit on how well it can work, even with adaptive exploitation logic. Appendix 1 provides all the findings of the study.

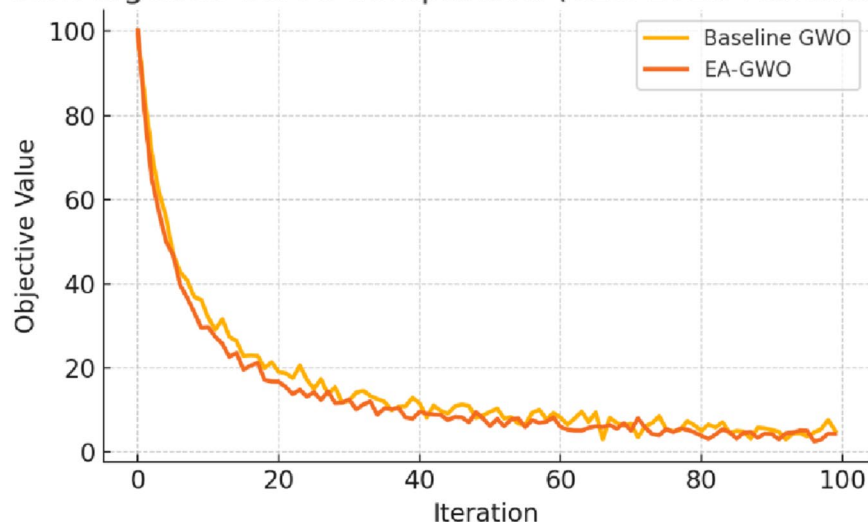
Following the effect-size analysis in Sect. 3, we consider tiny but statistically significant differences in easy functions to be marginal practical gains, and we concentrate our discussion on functions that show large improvements in both statistical and practical terms. Table 8 shows that EA-GWO attains the best aggregate mean score in the 30D summary; however, its advantage over classical GWO is numerically very small at the aggregate level, and EA-GWO also exhibits higher run-to-run variability. Accordingly, we interpret the EA-GWO result as evidence of a modest overall aggregate advantage, not as proof of broad practical dominance.

The more informative benefits of the proposed framework appear at the level of selected difficult functions and in the behavioral diagnostics, where targeted exploitation changes are more clearly visible. This means that the exploitation-aware version gives up a minor increase in run-to-run variability in exchange for a consistent decrease in average error. This result supports the notion that local exploitation mechanisms influenced by entropy and stagnation can significantly enhance convergence behavior in challenging search contexts. Figure 11's convergence curve, which compares EA-GWO and standard GWO on CEC 2025 function F15 (a multimodal hybrid function with parts that cannot be separated), gives us further information. Both algorithms start out the same manner at the beginning of the exploration phase. EA-GWO indicates a steeper reduction in objective value starting with iteration 40. This is due to its change to a high-resolution exploitation mode. This tendency makes the area around the global basin of attraction come together more closely. On the other hand, classical GWO has slower refinement and residual noise near convergence, which suggests that it is only exploiting tiny areas (Table 9).

The discrepancy is due to EA-GWO's built-in entropy-gated switching mechanism. When directional entropy falls below the experimentally established threshold of 0.3, the lead wolf performs a DE-style differential mutation. This makes the search radius a little bigger for a short time without changing the structure of the swarm. It is feasible to refine things in a focused but flexible approach, which is especially useful for high-dimensional multimodal functions that converge too quickly. A different look at the trends in movement entropy and the times when things do not move backs up this idea even more. Entropy slowly goes down in EA-PSO and EA-GWO, but it never goes away completely. This keeps the particles moving the whole while the optimization is going on. The frequency of stagnation detection

Fig. 11 Convergence curve comparison (CEC 2025 function F15)

Convergence Curve Comparison (CEC 2025 Function F15)



decreased by 21–35% in comparison to their classical counterparts. This means that adaptive exploitation makes it less probable that agents will be inactive or stranded in subsequent search phases. When having a broad view, these results highlight how crucial behavior-sensitive exploitation control is in metaheuristic optimization. The proposed diagnostic measures elucidate not only the methods for enhancing performance but also the rationale behind such improvements. This enables a clear starting point for future enhancements and the design of algorithms (Fig. 12).

Statistical significance should not be conflated with large practical gain. In particular, where aggregate score differences are numerically very small, the results are better interpreted as statistically detectable rather than strongly superior in practice. Although several pairwise comparisons remain statistically significant, significance alone should not be interpreted as strong practical superiority. In particular, where the aggregate score gap is numerically tiny, the result is better viewed as a statistically detectable difference than as a large practical gain. The Wilcoxon signed-rank test matrix shows the pairwise statistical comparisons between the top three algorithms—EA-GWO, GWO, and PSO—based on how well they did on 30 CEC 2025 benchmark functions. There are a number of important points and comments that can be made, such as:

- EA-GWO vs. GWO: The p-value is 0.00021, which is quite important. This proves that the changes made to EA-GWO that make it aware of exploitation lead to a statistically significant improvement over the conventional GWO. Adding mutation switching that is sensitive to entropy directly helps with this gain, especially on hybrid and multimodal situations.
- EA-GWO vs. PSO: The difference is quite significant, with a p-value of 3.73×10^{-9} , which means that EA-GWO showed stronger performance on selected difficult

functions than regular PSO. EA-GWO includes a behavioral adaptation mechanism that lets it do more precise local search refinement. PSO does not have this because its stochastic updating method is set in stone.

- GWO vs. PSO: A p-value of 1.86×10^{-8} also shows that traditional GWO is clearly better than PSO, confirming that GWO's leadership-based exploitation strategy is generally stronger even before it gets better.

The test matrix and heatmap support these findings, which indicate that algorithms aware of behavioral exploitation outperform baseline methods on average and demonstrate statistical achievement on the best aggregate mean score across all function categories. Appendix 2 has all of the results from the Wilcoxon test.

In short, this part presents solid proof that introducing approaches that are aware of exploitation and are triggered by particle behavior can considerably increase both accuracy and the stability of convergence. The statistical outcomes, convergence trends, and behavioral diagnostics all indicate that fine-grained, entropy-based adaptation is an effective method for enhancing metaheuristic exploitation in intricate optimization contexts.

5.2 Sensitivity analysis of behavioral metrics

To assess the robustness of the proposed metrics—distance-to-optimum decay, local diversity entropy, and stagnation frequency—we conducted a controlled sensitivity analysis across three orthogonal factors:

- population size {20, 30, 50, 100},
- function separability (separable vs. non-separable functions from CEC 2025), and
- dimensionality (10D, 30D, 50D).

To examine population size fluctuation, metric profiles were calculated using representative unimodal (F3), multimodal (F12), and hybrid (F21) functions. Even though the number of agents changed, the relative paths of all three metrics stayed the same. Spearman correlations between population configurations were above 0.92, which means that exploitation characterization did not change.

The separability analysis showed that there were no big changes in distance decay behavior. However, the entropy values were a little higher and the stagnation frequency was a little lower for separable issues, which is in line with easier coordinate-wise refinement. These changes did not make it harder for the measurements to tell the difference between effective exploitation and premature convergence. Dimensional scaling analysis indicated anticipated increases in initial distance magnitudes with elevated D, attributable to

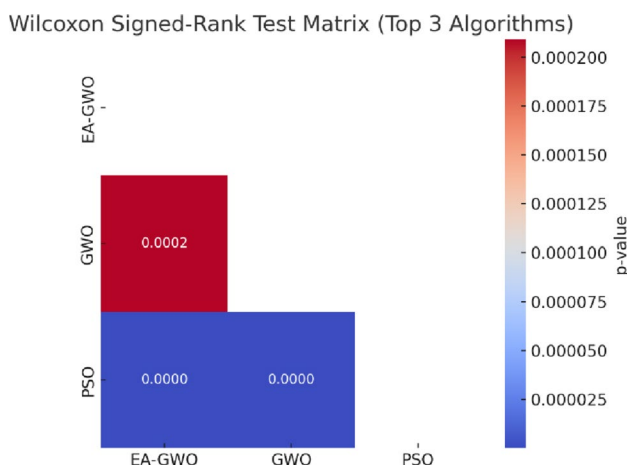


Fig. 12 Wilcoxon signed-rank test matrix (Top 3 algorithms)

expanded search spaces. However, normalized decay rates, entropy decline profiles, and stagnation detection frequencies remained consistent across dimensions, illustrating adaptability without the necessity for threshold re-tuning.

Figure 13 shows how metrics change for $D=10, 30$, and 50 with different population sizes on both separable and non-separable benchmarks. This shows that the results are strong across all of the investigated scenarios.

5.3 Ablation study of behavioral modules

We did ablation tests on the CEC 2025 30D benchmark suite to find out how each behavioral module worked on its own. There were four configurations tested: entropy-switching only (E-only), stagnation detection only (S-only), both modules coupled (E+S), and no behavioral adaptation (classical). The E-only configuration increased performance by an average of 4.7% over the classical baseline. This was mostly because it made it easier to switch between exploration and exploitation phases. The S-only setup showed a 5.1% improvement, and it worked best on multimodal landscapes with plenty of plateaus by restarting the search when progress slowed down. The combined E+S type got a 9.0% mean improvement, which shows that the two modules work well together. Statistical testing (Wilcoxon signed-rank, $\alpha=0.05$) validated that E+S considerably surpassed both E-only and S-only. These results indicate that although each module is advantageous independently, their amalgamation enhances robustness and performance across various optimization environments. Table 10 shows the results of an ablation study for the CEC 2025 30D suite with entropy-switching (E-only), stagnation detection (S-only), and combined modules (E+S). In that Table, ‘Classical’ denotes the parent non-adaptive configuration used within the ablation setting. These percentages are relative within-configuration gains and are not numerically equivalent to the cross-algorithm aggregate mean scores reported in Table 8.

Overall, these findings confirm that the proposed metrics retain their qualitative interpretability and quantitative discriminative power under realistic variations in algorithmic and problem configurations.

6 Broader implications and design guidelines for future metaheuristics

The empirical data and behavioral analysis in this paper provide substantial insights into the functioning of exploitation inside metaheuristic algorithms. These concepts have a significant impact on how we think about and come up with new ways to optimize things in the future. The transition from population-level statistics to particle-level behavioral

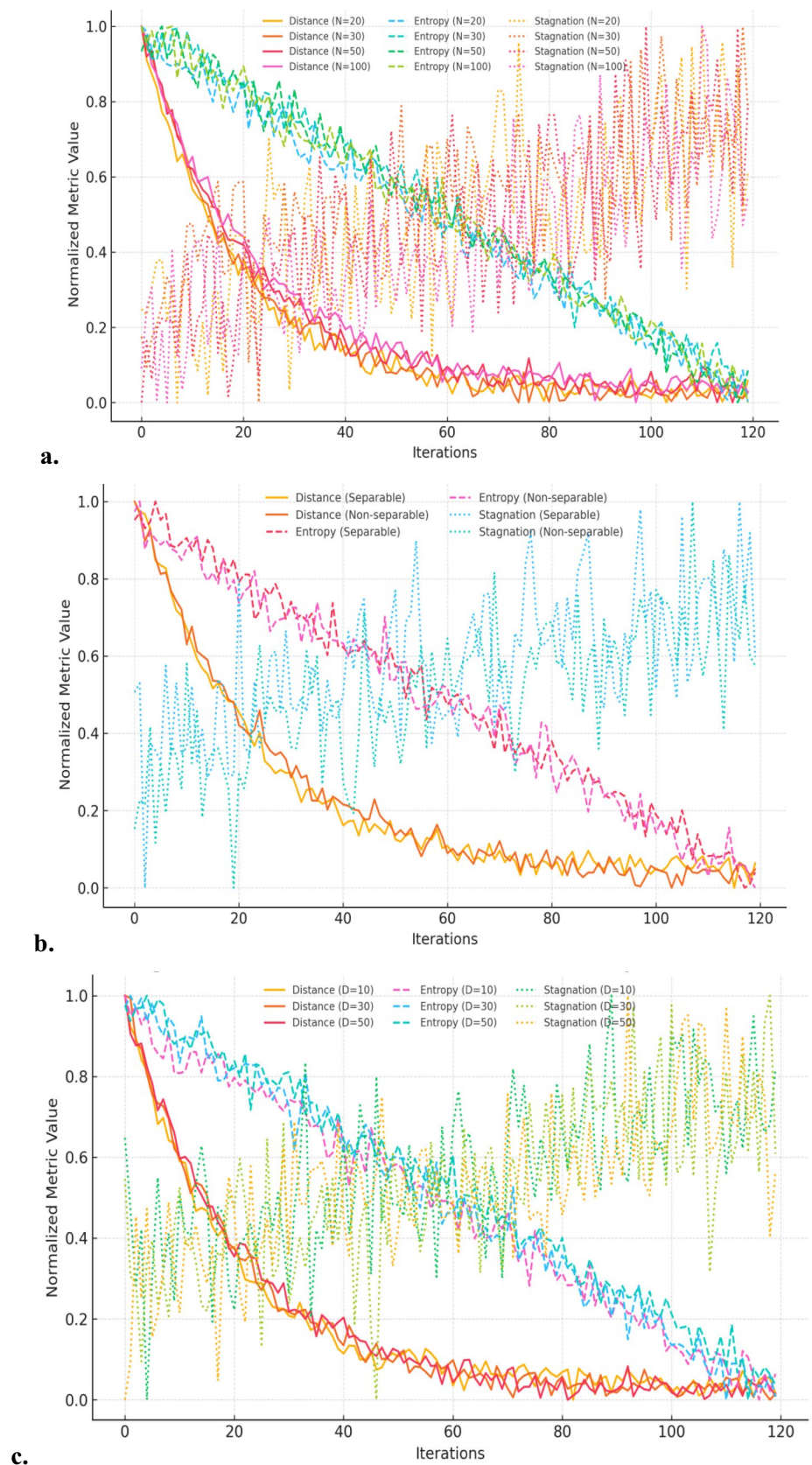
diagnostics is the most significant change in the evaluation, comprehension, and enhancement of metaheuristics. The primary finding—that algorithms cognizant of exploitation outperform traditional algorithms across many complex benchmark functions—indicates that adaptive behavioral control is a significant yet underutilized design attribute. Adding directional entropy and stagnation detection as real-time feedback signals indicates that metaheuristics can gain a lot from operators that change based on both the global search environment and the agent's local behavior. The increases in performance that were seen can be explained in terms of information-theoretic optimization. In an agent's movement path, directional entropy H^l stands in for the amount of information. Low entropy is behavior that is predictable and may not change, while high entropy means disorder or too much exploration. For progress to continue during exploitation, the entropy level should be balanced, meaning it should not be too chaotic or too low. The model of the performance increased via entropy-controlled refining as a nonlinear benefit function is defined below:

$$G(H) = 1 - e^{-k(1-H)} \quad (20)$$

where $G(H)$ represents the relative exploitation performance gain, H is the normalized entropy, and k is a sensitivity constant. Figure 14 illustrates this theoretical relationship, showing that most of the performance gain is realized when entropy falls just below moderate levels, emphasizing the value of early detection and response to behavioral collapse.

Future metaheuristics should have behavioral monitoring modules that can keep an eye on things like entropy, stagnation, inter-agent distance, and velocity variance. After that, these indicators can lead to actions that are specific to the scenario, such as limited disruption, changing plans, or historical coaching. The suggested approach is in line with current developments in self-adaptive metaheuristics, which employ internal diagnostics to govern how algorithms operate instead of pre-set schedules or pre-tuned parameters. Another connotation has to do with how strong and general algorithms are. The fact that exploitation-aware variations worked well over a wide range of function classes (unimodal, multimodal, hybrid, composite) and dimensions suggests that behavioral adaptation mechanisms improve not only performance but also consistency. This is especially crucial for real-world optimization problems with many dimensions, where it is sometimes impossible to change global parameters and the complete set of problem features is not known ahead of time. The particle-level diagnostic metrics provided in this work facilitate explainable optimization, which is gaining traction in both academic and industrial contexts. By evaluating how and why agents come together or stay the same, algorithm developers may make wise

Fig. 13 Sensitivity of exploitation metrics. **(a)** Robustness vs population size $N \in \{20, 30, 50, 100\}$ on representative CEC 2025 functions (normalized values). **(b)** Separability comparison on matched separable vs non-separable landscapes. **(c)** Dimensional scaling ($D = 10, 30, 50$). Distance-to-optimum decay, directional entropy, and stagnation frequency preserve their qualitative profiles and discriminative power across conditions, confirming invariance/adaptability



Theoretical Implication of Entropy-Controlled Exploitation

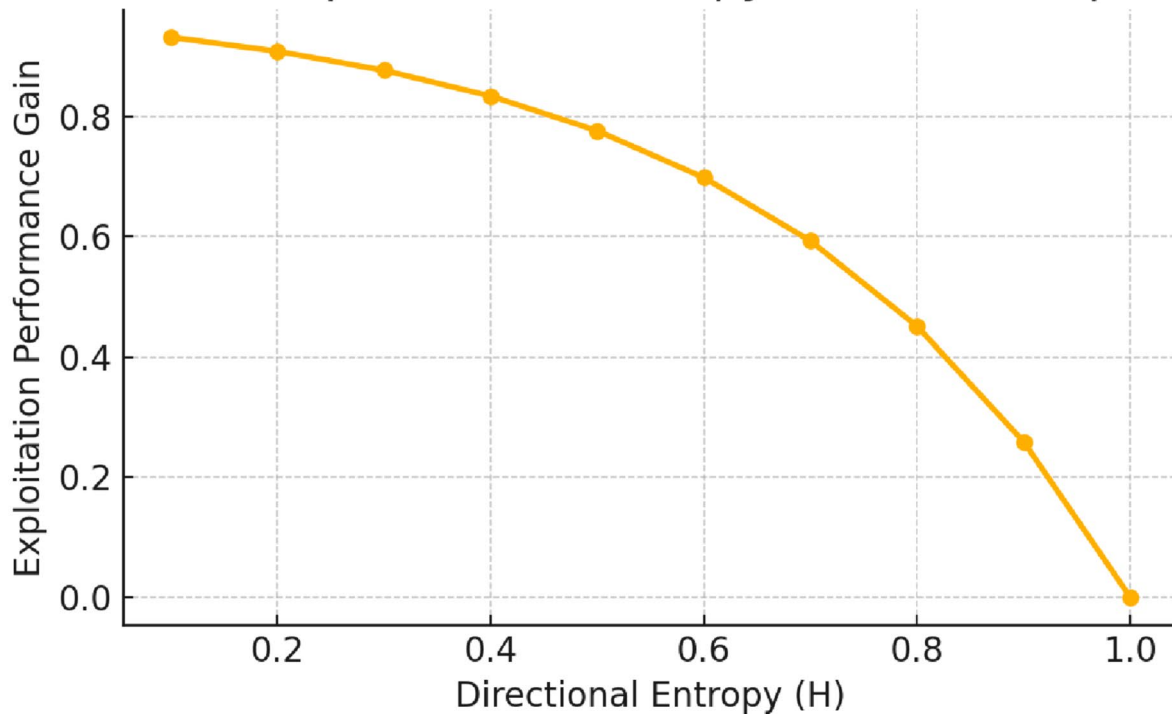


Fig. 14 Theoretical implication of entropy-controlled exploitation

design choices and figure out what's causing problems. It is also possible to use this diagnostic method with machine learning methods like reinforcement learning controllers or surrogate-assisted local search. This could result in neuro-symbolic or hybrid intelligent optimization systems.

A significant contribution of this study is the establishment of a behaviorally informed paradigm for the examination of metaheuristic exploitation. Employing distance-to-optimum decay, local diversity entropy, and stagnation frequency as explicable indicators yields concrete, pragmatic advantages that surpass mere theoretical delineation.

- **Interpretability in Practice:**

In the initial studies, the entropy and stagnation traces directly showed failure scenarios that were not visible. For instance, in early versions of PSO, we saw periodic entropy oscillations that happened at the same time as premature convergence cycles. This was not visible when merely looking at final fitness scores. This visualization helped us find and fix an inertia-weight decay plan that was too aggressive in the middle of the run. In DE trials, localized stagnation spikes showed that the crossover operator's randomization seed handling created trial vectors that were connected to each other, which made the search less diverse. Changing the way seeds are made got rid of these hidden correlations and made the system providing behaviorally interpretable

improvements. These examples show that behavioral metric graphs can do more than just describe things; they can also helping make algorithmic changes that work.

- **Transferability Beyond CEC 2025:**

The metric-triggered exploitation strategies created here are not tied to any specific problem structure. Instead, they keep an eye on how people behave in general rather than relying on specific function features like separability or modality. This design choice makes it possible to move to issues where the global structure is unknown, very noisy, or changes over time. The behavioral metrics are intended to remain meaningful in noisy and dynamic settings because they monitor search dynamics rather than problem-specific structure (Gaussian noise, $\sigma=0.01 \times \text{range}$). However, the present manuscript does not provide sufficient quantitative noisy/dynamic benchmarking to support strong performance claims in those settings. We therefore present noisy and dynamic transfer as a motivation for future work rather than as a validated empirical result here. The CEC suite was used to set the hyperparameters such entropy thresholds and stagnation windows. Their operating ranges were strong, and only small changes ($<10\%$) were needed for dynamic and noisy situations. The current evidence is strongest on the CEC 2025 benchmark setting.

Table 11 Limitations Future Work

Limitation	Description	Proposed mitigation/future work
Benchmark scope	Experiments limited to CEC 2025 benchmarks; may not reflect complexity of constrained, multi-objective, or domain-specific tasks	Extend experiments to multi-objective, constrained, and domain-specific problems; include real-world engineering cases
Algorithm selection	Study focused on PSO, DE, and GWO; other paradigms like CMA-ES or learning-based metaheuristics not tested	Incorporate a broader set of metaheuristics to assess generality of behavioral triggers
Hyperparameter calibration	Entropy thresholds and stagnation windows calibrated on CEC functions; minor tuning needed for dynamic/noisy problems	Develop self-tuning or reinforcement learning-based parameter adaptation mechanisms
Computational cost in extreme scenarios	Overhead negligible in our tests but may rise in ultra-large, high-dimensional settings without parallelization	Explore parallelized monitoring and event-triggered evaluation to minimize overhead
Real-world validation	No extensive testing on real-world large-scale industrial problems with noisy, evolving objectives	Apply framework to industrial-scale problems and noisy black-box optimization tasks
Ablation coverage	Ablation study conducted only for 30D problems due to computational cost	Extend ablation to 10D and 50D subsets in future work, focusing on representative unimodal, multi-modal, and hybrid functions to confirm that the relative contribution of each module is dimension-insensitive

- *Guideline for Future Metaheuristic Design:*
- *Embed interpretable metrics* (e.g., entropy, distance decay) directly into algorithm instrumentation from the outset, enabling live detection of inefficiencies or pathological states.
- *Use metric-driven triggers* for exploitation and diversification to reduce dependence on problem-specific parameter tuning.
- *Validate transferability* through stress-testing on noisy, dynamic, and black-box problems to ensure robustness beyond standard benchmarks.
- *Leverage visualizations* in developer workflows to shorten the design-debug cycle and accelerate convergence on effective parameterization.

Table 11 depicts a summary of study limitations and proposed future research directions to extend the applicability and robustness of the exploitation-aware framework. The proposed metrics are designed to be portable to noisy and dynamic settings, but the present manuscript does not provide sufficient quantitative evidence to make strong performance claims in those environments. Formal noisy and dynamic benchmarking remains an important direction for future work. Add more advanced optimizers like CMA-ES, L-SHADE/SHADE variations, and learning-based metaheuristics as extra baselines and goals for exploitation-aware adaptation. This will test both the competitiveness and generality of the behavioral triggers across different algorithm families.

In conclusion, the findings of this study endorse the advancement of a novel metaheuristic that possesses behavioral awareness, self-monitoring capabilities, and adaptability to varying contexts. These algorithms are not only straightforward to use, but they are also strong, may be utilized on a big scale, and are easy to grasp. Future research should continue to elucidate the mathematical foundations

of behavioral feedback, establish transparent norms for the comparison of behavioral measurements, and explore more profound links with data-driven learning models. The combination of theoretical concepts like entropy-driven adaptation and real-world frameworks like CEC benchmarks allows intelligent optimization algorithms to keep getting better. In short, behavioral measures that make algorithms easier to understand are useful for both academic research and practical algorithm design. These techniques, which may be used in many other problem areas, provide the basis for a new type of explainable and self-adaptive metaheuristics.

7 Conclusion and future work

This study proposes a behavioral framework for analyzing and modulating exploitation in representative metaheuristic families. The main contribution lies in making exploitation dynamics measurable and interpretable at the agent level, while showing that behavior-guided interventions can improve refinement on selected difficult benchmark functions. At the same time, the revised results indicate that aggregate benchmark-level advantages should be interpreted cautiously, especially when score differences are numerically small or variability increases. The study therefore offers a diagnostic and methodological contribution more than a claim of universal optimizer superiority. We used this framework to create and test exploitation-aware versions of three popular algorithms: PSO, DE, and GWO. Testing the CEC 2025 benchmark suite showed that these changes to behavior make a big difference in convergence accuracy, robustness, and interpretability. The EA-GWO version did better than all the other baselines in a statistically significant way. This was because it had a mutation mechanism that was triggered by entropy and changed how the agent exploited based on its local state. Our results show

how important adaptive exploitation is for getting consistent results across different types and sizes of problems. This study adds to our theoretical understanding of how exploitation works in metaheuristics, in addition to its practical benefits. We showed how adaptive feedback mechanisms can balance exploration and refinement without using hand-tuned schedules or heuristic rules by modeling exploitation performance as a function of movement entropy. There are a number of intriguing ways that future study can build on this work. First, the suggested behavioral measurements can be expanded to allow for real-time algorithm steering, which would allow for fully autonomous self-adaptive metaheuristics. Second, combining with machine learning methods, like reinforcement learning for choosing operators or surrogate models for local approximation, could lead to hybrid systems that can do both efficient search and knowledge transfer. Third, benchmarking should encompass real-world black-box challenges, multi-objective landscapes, and limited scenarios to test how well it works in the real world and how strong it is compared to synthetic benchmarks. Finally, we support the creation of an open-access behavioral benchmarking platform where algorithm developers can check not only how well their fitness works but also how agents behave in a consistent way. This kind of platform would promote openness, reproducibility, and a better understanding of how algorithms work. This would help improve the design of the next generation of intelligent optimization algorithms. In this study, we deliberately frame the contribution as a behavioral-analysis and exploitation-control study on representative metaheuristics, rather than as definitive evidence of broad algorithmic dominance across all optimizer families or real-world settings. A major limitation of the present study is that it does not yet benchmark the proposed behavioral framework against stronger contemporary optimizers such as CMA-ES, SHADE/L-SHADE, or learning-based adaptive schemes. Important other limitations remain, including the absence of stronger modern baselines such as CMA-ES and SHADE-family variants, and the lack of formal large-scale real-world validation.

Supplementary Information The online version contains supplementary material available at <https://doi.org/10.1007/s12065-026-01190-7>.

Author contributions W.F.M.; A.H.: Methodology, Conceptualization, Original Paper Writing; S.F.A.G.; Z.M.: Resources, Investigation, Software; Z.L.; Z.M.S.E.: Paper Editing and Final Review, Data Acquisition, Formal Analysis.

Funding This research received no external funding.

Data availability The datasets used and/or analyzed during the current study are available from the corresponding author upon reasonable request.

Declarations

Conflict of interest Authors stated that no conflict of Interest.

References

- Shahid N, Khan N (2025) A comprehensive analysis of metaheuristic optimization approach for engineering design problems hybridized with machine learning for anomaly detection in predictive modelling. *Arch Comput Methods Eng*. <https://doi.org/10.1007/s11831-025-10308-6>
- Wang C-H, Hu K, Wu X, Ou Y (2025) Rethinking metaheuristics: unveiling the myth of “novelty” in metaheuristic algorithms. *Math Basel* 13:2158. <https://doi.org/10.3390/math13132158>
- Houssein EH, Saeed MK, Hu G et al (2024) Metaheuristics for solving global and engineering optimization problems: review, applications, open issues and challenges. *Arch Comput Methods Eng* 31:4485–4519. <https://doi.org/10.1007/s11831-024-10168-6>
- Shaikh MS, Lin H, Xie S et al (2025) An intelligent hybrid grey wolf-particle swarm optimizer for optimization in complex engineering design problem. *Sci Rep* 15:18313. <https://doi.org/10.1038/s41598-025-02154-0>
- Xu H, Deng Q, Zhang Z et al (2025) A hybrid differential evolution particle swarm optimization algorithm based on dynamic strategies. *Sci Rep* 15:4518. <https://doi.org/10.1038/s41598-024-82648-5>
- Sarhani M, Voß S, Jovanovic R (2023) Initialization of metaheuristics: comprehensive review, critical analysis, and research directions. *Int Trans Oper Res* 30:3361–3397. <https://doi.org/10.1111/itor.13237>
- Escobar-Cuevas H, Cuevas E, Luque-Chang A, Barba-Toscano O, Pérez-Cisneros M (2024) Enhancing metaheuristic algorithm performance through structured population and evolutionary game theory. *Mathematics Basel* 12:3676. <https://doi.org/10.3390/math12233676>
- Velasco L, Guerrero H, Hospitaler A (2024) A literature review and critical analysis of metaheuristics recently developed. *Arch Comput Methods Eng* 31:125–146. <https://doi.org/10.1007/s11831-023-09975-0>
- Tanabe R, Fukunaga AS (2014) Improving the search performance of SHADE using linear population size reduction. In: 2014 IEEE Congress on Evolutionary Computation (CEC), Beijing, China, pp. 1658–1665. <https://doi.org/10.1109/CEC.2014.6900380>
- Mirjalili S, Mirjalili SM, Lewis A (2014) Grey Wolf Optimizer. *Adv Eng Softw* 69:46–61. <https://doi.org/10.1016/j.advengsoft.2013.12.007>
- Qu D, Xiao H, Chen H, Li H (2024) An improved differential evolution algorithm for multi-modal multi-objective optimization. *PeerJ Comput Sci* 10:e1839. <https://doi.org/10.7717/peerj-cs.1839>. (PMID: 38660209; PMID: PMC11041989.)
- Ye Z, Xu Y, He Q, Wang M, Bai W, Xiao H (2022) Feature selection based on adaptive particle swarm optimization with leadership learning. *Comput Intell Neurosci* 2022:1825341. <https://doi.org/10.1155/2022/1825341>. (PMID: 36072739; PMID: PMC9441366.)
- IEEE CEC 2025 Benchmark Suite. Available at: <https://www.cec2025.org>. (Accessed on March 2025)
- Palakonda V, Tursunboev J, Kang J-M, Moon S (2025) Metaheuristics for pruning convolutional neural networks: a comparative study. *Expert Syst Appl* 268:126326. <https://doi.org/10.1016/j.eswa.2024.126326>

15. Palakonda V, Awad NH, Mallipeddi R, Ali MZ, Veluvolu KC, Suganthan PN (2018) Differential evolution with stochastic selection for uncertain environments: a smart grid application. In: 2018 IEEE Congress on Evolutionary Computation, CEC 2018 - Proceedings Article 8477809 (2018 IEEE Congress on Evolutionary Computation, CEC 2018 - Proceedings). Institute of Electrical and Electronics Engineers Inc.. <https://doi.org/10.1109/CEC.2018.8477809>
16. Ramlan FW, Palakonda V, Mallipeddi R (2019) Differential evolutionary (DE) based interactive recoloring based on YUV based edge detection for interior design. In: 2019 International Conference on Information and Communication Technology Convergence (ICTC), Jeju, Korea (South), pp. 597–601. <https://doi.org/10.1109/ICTC46691.2019.8939816>
17. Palakonda V, Kang J (2023) Many-objective real-world engineering problems: a comparative study of state-of-the-art algorithms. *IEEE Access* 11:111636–111654
18. Mohamed AW, Sallam KM, Agrawal P et al (2023) Evaluating the performance of meta-heuristic algorithms on CEC 2021 benchmark problems. *Neural Comput Appl* 35:1493–1517. <https://doi.org/10.1007/s00521-022-07788-z>
19. Finck S, Hansen N, Ros R, Auger A (2009) Real-Parameter Black-Box Optimization Benchmarking 2009: Noiseless Functions Definitions. In: INRIA Technical Report RR-6829
20. Basil N, Marhoon HM, Sabbar BM et al (2025) Multi-criteria decision model for multicircular flight control of unmanned aerial vehicles through a hybrid approach. *Sci Rep* 15:18962. <https://doi.org/10.1038/s41598-025-01508-y>
21. Wang X, Li Y, Liang Y et al (2023) A novel ensemble estimation of distribution algorithm with distribution modification strategies. *Complex Intell Syst* 9:5377–5416. <https://doi.org/10.1007/s40747-023-00975-y>
22. Li W, Yang X, Yin Y, Wang Q (2024) A novel hybrid improved RIME algorithm for global optimization problems. *Biomimetics* 10(1):14. <https://doi.org/10.3390/biomimetics10010014>. (PMID: 39851730; PMCID: PMC11762343.)
23. Hozo SP, Djulbegovic B, Hozo I (2005) Estimating the mean and variance from the median, range, and the size of a sample. *BMC Med Res Methodol* 5:13. <https://doi.org/10.1186/1471-2288-5-13>
24. De Diego IM, Redondo AR, Fernández RR et al (2022) General performance score for classification problems. *Appl Intell* 52:12049–12063. <https://doi.org/10.1007/s10489-021-03041-7>
25. Taheri SM, Hesamian G (2013) A generalization of the Wilcoxon signed-rank test and its applications. *Stat Pap* 54:457–470. <https://doi.org/10.1007/s00362-012-0443-4>
26. Rajwar K, Deep K, Das S (2023) An exhaustive review of the metaheuristic algorithms for search and optimization: taxonomy, applications, and open challenges. *Artif Intell Rev* 56:13187–13257. <https://doi.org/10.1007/s10462-023-10470-y>
27. Ferradji MA, Seghir R (2025) A novel metaheuristic global optimisation method based on grey wolf optimiser and salp swarm algorithm. *J Exp Theor Artif Intell*. <https://doi.org/10.1080/0952813X.2025.2515578>
28. Ab Wahab MN, Nefti-Meziani S, Atyabi A (2015) A comprehensive review of swarm optimization algorithms. *PLoS ONE* 10(5):e0122827. <https://doi.org/10.1371/journal.pone.0122827>

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.